



Procedural Generation of a Destructible Voxel Environment in Unreal Engine 5

Final Year Project Report
Academic Year 2025 – 2026

Student Name:	Abhijit Shidlagatta
Student Email:	Abhijit.Shidlagatta@citystgeorges.ac.uk
Student Number:	210012420
Degree Programme:	BSc Computer Science
Module Code:	IN3007
Consultant:	Mr Atif Riaz
Project Proposed by:	Self

Main Body Word Count $\approx$ 10,300 words

April 12, 2026

Abstract

A big challenge in modern game development is trying to balance procedural generation with real-time environmental destructibility. Most modern games generate a static level or generate seed-based environments upon initialisation, which inherently limits replayability. On the other hand, fully destructible games are often empty sandboxes that incur severe computational overhead when rapid generation is attempted. Thus, this project tries to bridge and fix this gap by building an "on-demand" procedural dungeon generator using C++ and Unreal Engine 5.

A custom voxel system that lets the player destroy terrain (carve dungeon rooms) in real-time was implemented. To prevent the game from freezing when the CPU tries to do all that heavy math, Axis-Aligned Bounding Box (AABB) checks were implemented and the heavy computational lifting was offloaded to background threads using Unreal's async tasks. A full game loop was then built around it, including a data-driven shop economy and enemies that use a Finite State Machine (FSM). The performance tests show that the multithreaded code actually keeps the frame rate stable without leaking memory. To conclude, this project proves that using async C++ to work with voxels is a highly viable way to make dynamic games that do not crash and allows for a replayable and smooth experience for this dungeon room-carving procedurally generated game.

Acknowledgements

I would like to thank my project supervisor, Mr. Atif Riaz, for his constant guidance, feedback and support throughout the development of this project. He did this by providing keen insights, showing interest in my project and helping me by reviewing my final report to ensure it stayed professional and high-quality.

I ensured that during all of our conversations, reviews and meetings, I took meticulous notes to ensure I considered and implemented all of his feedback.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Problem Description	1
1.2 Project Objectives	2
1.3 Anticipated Beneficiaries	3
1.4 Outline of Work Performed	3
1.5 Assumptions and Scope Limitations	3
1.6 Changes from the Project Definition Document	4
1.7 Report Structure	5
2 Output Summary	6
2.1 Output 1: Voxel Layout Generator Prototype	6
2.2 Output 2: Custom C++ Codebase	6
3 Context	8
3.1 Application Domain	8
3.2 Existing and Related Work	8
3.3 Relevant Technologies and Methods	10
4 Method	12
4.1 Software Engineering Approach	12

4.1.1	Justification of Agile Methodology	12
4.1.2	Mathematical Justification of Voxel Architecture	12
4.1.3	Iterative Phases	13
4.2	Implementation and Iterative Development	14
4.2.1	Iteration 1: Spatial Validation and AABB Collision	14
4.2.2	Iteration 2: Advanced Geometric Carving and Materials	16
4.2.3	Iteration 3: Organic Cave Tunnelling	17
4.2.4	Iteration 4: Asynchronous Voxel Modification (Multithreading)	18
4.2.5	Iteration 5: Data-Driven Architecture and UI Interoperability	19
4.3	Testing and Evaluation Strategy	20
4.3.1	Algorithmic Verification (White-Box Testing)	20
5	Results	22
5.1	System Architecture and Design	22
5.1.1	High-Level Architecture and Class Design	22
5.1.2	Economy Domain and Data-Driven Architecture	24
5.1.3	Artificial Intelligence Design Pattern	25
5.2	Implementation Results	26
5.2.1	AI Navigation and Combat Adaptation	26
5.2.2	Gameplay Systems and Combat Loop Evaluation	27
5.3	Testing and Performance Results	28
5.3.1	Hardware Test Environment	28
5.3.2	Multi-threading Execution Times and Thread Overhead	29
5.3.3	GPU Rendering and Occlusion Culling	30
5.3.4	Memory Consumption and Octree Scalability	30
5.3.5	Algorithmic Reliability and AABB Fallback Success	33
5.3.6	Frame-Time Consistency and Stutter Mitigation	33
6	Conclusions and Discussion	34
6.1	Objectives Review	34

6.2	Discussion of Findings	34
6.3	Implications for Future Work	35
6.4	Personal Reflection	36
6.5	Legal, Social, Ethical and Professional Issues	36
6.5.1	Project Category	36
6.5.2	Legal Considerations and Copyright	36
6.5.3	Privacy and Data Protection	37
6.5.4	Potential Harms and Risk Mitigation	37
6.5.5	Professional Practice	38
References		39
A Project Definition Document		42
B Reuse Summary		44
C Core C++ Code Snippets		45
C.1	AABB Spatial Validation Logic	45
C.2	Asynchronous Voxel Carving Loop	46
D Additional Product Screenshots		47
E Test Plans and Results		52
F User and Installation Guide		53
G Generative AI Declaration		54

List of Figures

4.1	Debug visualisation of an edge-case spatial validation failure. Using the Euclidean distance check for a cuboid volume failed to account for the corners. . .	14
4.2	Early development iteration demonstrating the algorithmic failure. Without robust spatial validation, the system decided to generate new rooms directly inside existing ones if the player aimed the wall-breaking spell at an angle towards the corners or edges of the cube, breaking the dungeon's topology. . . .	15
4.3	Diagram explaining the AABB spatial validation fallback logic.	16
4.4	Triplanar material projection logic, implemented to prevent texture stretching on dynamically carved voxel geometry.	17
4.5	Early iteration of the fallback tunneling logic. The algorithm failed to calculate the intersection point between existing caves, resulting in dead-end tunnels. . .	18
4.6	White-box testing using Unreal Engine's debug drawing functions. The wire-frame boxes allows for visual verification of the AABB bounds and spatial querying logic. The Blue outline represents the space in which the tunnel can be formed or if needed where a possible fallback could occur.	21
5.1	UML Class Diagram of the core C++ system architecture.	23
5.2	The data-driven architecture of the economy system using the Unreal Engine Data Tables.	24
5.3	Finite State Machine (FSM) describing the autonomous decision-making loop of the dungeon enemies.	25
5.4	An editor-perspective view of a fully generated network of varying-sized rooms showing the successful AABB spatial validation and tunnel connections. . . .	26
5.5	Visualisation of the Unreal Engine NavMesh generation failure. The dynamically generated green navigation bounds failed to cover the floor entire parameters and under the enemy's spawnpoint.	27
5.6	On-screen profiling of the underlying Voxel Data Octrees generated during runtime.	30
5.7	On-screen profiling of the physical memory allocation during runtime.	31

5.8	Physical memory execution usage monitored during dynamic rendering at different generated room quantity intervals, demonstrating stable allocation.	31
5.9	Scatter graph demonstrating a linear increase of Voxel Data Octrees relative to generated room quantity.	32
D.1	In-game player perspective demonstrating the dynamic UI, Crosshair states, and a procedurally carved spherical dungeon room.	47
D.2	In-game player perspective showing caves tunneling to different rooms based on the player's choice whenever their spell is ready.	48
D.3	Blueprint Event Graph detailing the frontend transaction logic for the interactive Shopkeeper system and when the refresh inventory function gets called.	48
D.4	Blueprint logic for pulling variables from the C++ <code>DungeonCharacter</code> class to dynamically update the player's stat screen.	49
D.5	Blueprint logic for total damage dealt to be shown shown on <code>PlayerCharacter</code> 's defeat screen.	49
D.6	Frontend logic for spawning and animating floating damage numbers during combat interactions to help visualise changes in damage stats when player buys items or when the stats get recalculated.	50
D.7	Ingame view of the interactive Shopkeeper menu, displaying available equipment and items.	50
D.8	Ingame view of the player's Inventory screen dynamically displaying current combat statistics, equipped items, collected materials, and kill counts/current gold for the current dungeon run.	51

List of Tables

5.1	Total execution time (ms) for generating 108,000 volumetric units.	29
5.2	Normalised execution time multipliers relative to the synchronous Game Thread baseline. Values > 1.00 indicate scheduling overhead, whilst values < 1.00 indicate a performance speedup.	29
5.3	Data structure scaling and memory allocation relative to generated rooms. (The base total Memory used in the Unreal Editor itself without running anything was 2246.1 MB).	31
5.4	AABB spatial query resolution out of 100 generation attempts per condition. . .	33
E.1	White-Box Test Cases for Voxel Generation Logic	52

1. Introduction

This chapter introduces the problem of balancing structured procedural generation with real-time environmental destructibility. It outlines the project's core technical objectives, its beneficiaries, and the strict hardware and scope limitations defining its boundaries.

1.1 Problem Description

In modern game development, "Procedural Content Generation" (PCG) is the algorithmic creation of data such as 3D environments, textures, or rules - rather than relying on human designers to manually build every asset. However, current PCG applications typically tend to fall into two categories, both of which have significant limitations.

The first category consists of traditional room-based games (like the roguelike genre). In these games, generation only occurs once during the initial loading phase. The system selects pre-built 3D structures or rooms and stitches them together along with other features to form a map. Once the level loads, the environment's "topology" (meaning the physical spatial layout, structural connectivity, and available pathways) is completely static. A player cannot physically break down a solid wall to create a shortcut or force the game to generate a new room where the player actually wants most of the time. This results in the limitation of the player's physical freedom and agency.

The second category consists of fully destructible sandbox games, most famously *Minecraft* (Mojang Studios, 2011). These games rely on "voxels" to construct their worlds. While *Minecraft* visually renders these as blocky 3D cubes, a voxel in computer science is fundamentally just a data point on a massive 3D grid that stores volumetric information (for example storing a numerical value to indicate whether a specific coordinate is solid stone or empty air). Because the environment is stored as raw, editable data rather than a static, hollow 3D model, the player has absolute physical freedom to destroy or build terrain anywhere. However, the procedural generation in these massive sandboxes is strictly environmentally driven and pre-determined by the global seed. While the algorithm *does* generate complex, highly structured architecture (such as underground strongholds or villages), it does so passively as the player explores new areas of the map or loads new chunks. The player has no agency or freedom over the generation logic itself meaning that a player cannot strike a random cave wall and command the game engine to dynamically calculate, validate, and spawn a fully functional challenge room exactly at that location on demand.

The core problem this project addresses is bridging the technical gap between these two extremes: creating a structured procedural dungeon crawler where the player has the absolute physical freedom to carve brand-new, fully featured challenge rooms out of solid walls at any given time. This however comes with a multitude of problems that need to be solved in order to achieve this project goal:

- **Querying:** The game has to check a massive 3D database (the voxel grid) to see if there's enough space to build (or carve out) a new room.
- **Decision-Making:** It then has to decide whether to build a big room, a small tunnel, or something else entirely based on the information received from the space-check.
- **Carving:** It has to perform a huge, real-time edit on the voxel data to implement that decision made.
- **Pathfinding:** If it's making a tunnel, it has to run an algorithm to find the nearest existing room to allow the player to pathfind properly.

Doing all of this at once would most likely cause massive lag spikes especially in a full-release game with detailed assets and other sorts of data being rendered and calculated. If you run these calculations on the engine's main thread, the whole game freezes. On top of that, dynamically changing the terrain breaks standard AI navigation meshes (NavMeshes). My project will solve this by building a C++ prototype in Unreal Engine 5 that handles this entire "on-demand" process smoothly without crashing and maximising the performance statistics.

1.2 Project Objectives

The main objective of this project is to design and implement a voxel layout generator or director in C++ and Unreal Engine 5 that is capable of performing real-time procedural generation of new challenge-based rooms within a dynamic voxel world. This can be achieved by meeting these following sub-objectives:

1. Create a stable foundation for real-time C++ voxel manipulation that runs on background threads so it does not stall the game.
2. Develop a high-performance "Space Check" algorithm using Axis-Aligned Bounding Boxes (AABB).
3. Implement functional enemy AI capable of basic navigation, player pursuit, and attacks within the procedurally generated voxel grid based on the enemy type (such as flying type or melee type or boss type enemies).

4. Integrate the procedural generation mechanics into a functional game loop along with a player resource economy, inventory and a player stats system for upgrades, to prove it works as an actual game.

1.3 Anticipated Beneficiaries

As the project is a "proof of concept" prototype, the primary beneficiaries are:

- **Myself (the student):** I am the primary beneficiary since I will gain significant and in-depth practice and expertise in C++, Unreal Engine 5, procedural generation methods and voxel data structures.
- **Game developers:** This project's findings and open-source code if successful and published could serve as a feasible study for other game developers who wish to explore this game style.

1.4 Outline of Work Performed

The project was divided into four main phases:

- **Phase 1: Research and Setup** - Existing literature and research on voxel meshing and how multi-threading works in UE5 was first looked through.
- **Phase 2: Core Foundation** - Then the core C++ architecture was built by implementing the voxel plugin and the writing of the initial carving algorithms and code.
- **Phase 3: Gameplay Systems** - The enemies, Shopkeeper, the data-driven economy, and the logic to make the game actually playable such as when the player can generate new rooms from carving was implemented.
- **Phase 4: Evaluation** - UE5's profiling tools were used to stress-test the math. After doing so, it was packaged into a final Windows executable for submission.

1.5 Assumptions and Scope Limitations

Assumptions:

- **Hardware:** The player needs a modern multi-core CPU. The async background threads will not work well on a single-core machine, and the carving will therefore lag their PC.

- **Platform:** This is built for a standard Windows PC with a keyboard and mouse and does not have capabilities for controllers or other types of operating systems.

Scope Limitations:

- **Networking:** This is strictly a single-player game. Trying to sync millions of broken voxels over a server was out of scope and focus for this timeframe.
- **Graphics:** The C++ backend and math was the main focus of this project, so high-end 3D models and shaders were not included. Instead the base assets provided by the Unreal Engine content browser were used as placeholders.
- **AI Complexity and Tactical Behavior:** The implementation of artificial intelligence was deliberately restricted to a fundamental Finite State Machine (FSM) - governing direct pursuit, attack flash warning and basic attack patterns. Advanced tactical behaviors such as dynamic flanking, squad-based grouping, or using environmental cover were strictly excluded from the project's scope. In a traditional game environment with static geometry, AI agents can handle pre-computed spatial data (such as Unreal Engine's Environment Query System) to identify safe cover points or optimal tactical positioning. However, in a fully destructible voxel environment, the topology is highly unpredictable in nature. A solid wall providing cover can be instantly destroyed into a new empty space through a tunnel or a new dungeon room by the use of the player's Tunnelling Spell. Continuously recalculating valid cover points and tactical advantages in real-time alongside the $\mathcal{O}(n^3)$ voxel meshing updates would introduce severe computational bottlenecks. Solving this dynamic tactical analysis to adapt to the ever-changing environment is a massive concept or issue worthy of being undertaken as its own dedicated project. Consequently, restricting the AI to direct line-of-sight pursuit ensured that the core objective of handling the real-time procedural generation and asynchronous chunk updates could be thoroughly tested without the system crashing due to enemy logic or processor overload.

1.6 Changes from the Project Definition Document

- **Name Change:** Originally the main C++ class was called "Challenge Director", but it was renamed to "Voxel Layout Generator" because the code ended up focusing much more on the spatial math to improve the replayability and the player's freedom rather than managing the actual challenges involved.
- **Bigger Scope:** In the PDD I mentioned that I would spawn a single test enemy, however, this objective was easily achieved and therefore decided on expanding this massively to include a full working shop, floating damage numbers, multiple enemy types with varying

attack patterns and stats, and a whole stat upgrade system using enemy drops to really prove it works as a game both performance-wise and in terms of the player's enjoyment.

1.7 Report Structure

Chapter 2 shows the final deliverables. Chapter 3 goes over the background research that was partaken. Chapter 4 walks through how generator was coded/implemented and fixed bugs and issues discovered. Chapter 5 shows all the performance profiling data. Chapter 6 reviews the objectives and covers the Legal, Social, Ethical, and Professional Issues (LSEPI).

2. Output Summary

This chapter provides a concise list of the final deliverables produced during this project, detailing the software prototype and the C++ codebase involved.

2.1 Output 1: Voxel Layout Generator Prototype

Description: A packaged, playable Windows executable demonstrating real-time procedural dungeon generation with multi-threaded voxel carving, multiple enemy archetypes, and a functional player economy.

Type: Software application - Windows Executable (.exe).

Size: 1.43 GB.

Reused material: Built on Unreal Engine 5.4 using the Free Legacy Voxel Plugin (open-source under MIT license).

Recipient(s)/Users: University markers and game developers looking for a feasibility study on voxel mechanics.

Benefit: Proves you can successfully transition from static generation to dynamic voxel destruction without crashing the engine. Also proves that procedurally generated games can provide a high level of freedom and replayability for the user.

Results links: See Sections 5.2 and 5.3 in Chapter 5.

Location: The complete executable is in the official product package, with instructions in Appendix E.

2.2 Output 2: Custom C++ Codebase

Description: The underlying multi-threaded C++ architecture and spatial querying logic driving the Voxel Layout Generator.

Type: Software Source Code.

Size: Roughly 1,450 lines of custom C++ code I wrote across 10 core source files.

Reused material: Unreal Engine 5 API macros and header files from the Voxel Plugin.

Recipient(s)/Users: Myself (for my portfolio) and the university markers.

Benefit: Provides concrete proof of the backend engineering, memory management, and multi-threading implementation required to meet all the objectives.

Results links: See Section 5.2 in Chapter 5.

Location: Submitted as part of the product package, with a full directory summary provided in Appendix B.

3. Context

This chapter goes through and establishes the academic and technical background of the project by exploring the current state of Procedural Content Generation (PCG), analysing the existing commercial solutions, and outlining the core technologies (such as voxels and multi-threading) required to achieve real-time environmental manipulation.

3.1 Application Domain

My project's concept sits right in the middle of Procedural Content Generation (PCG) and voxel technology.

PCG means using algorithms to build game environments instead of having a human designer place everything by hand (Togelius et al., 2011; Shaker et al., 2016; Hendrikx et al., 2013). Usually, games use PCG to save memory and make the game replayable. But traditional PCG just snaps together static 3D models. It creates a different layout every time you play, but once the level loads, you cannot physically change the pre-determined environment.

To make the environment completely destructible or changeable on-demand, I had to work with voxels which are 3D pixels. Unlike normal polygons that just draw a hollow shell around an object, voxels store data about the volume inside the object (Lengyel, 2010). This is what makes real-time carving possible. My code can query a coordinate in 3D space and dynamically change it to "air" (Lorensen and Cline, 1987). Voxel data is commonly organised using an octree data structure, which recursively subdivides 3D space into eight child nodes to support efficient spatial queries while minimising memory overhead (Meagher, 1982; Laine and Karras, 2010). The issue is managing how heavy this is on the CPU without ruining or pausing the active game loop.

3.2 Existing and Related Work

To contextualise the gap this project addresses, it is necessary to examine how contemporary commercial games handle Procedural Content Generation (PCG). As Short and Adams (2017) note, implementing PCG that feels intentional requires a delicate balance between absolute algorithmic randomness and structured, designer-driven logic.

The most prevalent approach in the roguelike genre is room-based PCG, famously used in titles such as *Hades* (Supergiant Games, 2020) and *Dead Cells* (Motion Twin, 2018). These systems do

not generate geometry voxel-by-voxel. Instead, they rely on pre-authored, handcrafted modular "chunks". The procedural algorithm functions as a graph-based layout generator - randomly selecting and stitching these pre-defined rooms together at runtime. While this method guarantees a highly controlled gameplay pacing and ensures that AI NavMeshes can be pre-baked into the static geometry, it fundamentally restricts player freedom or agency. The environment remains entirely unalterable meaning the player cannot bypass a locked door or forge a new path by physically destroying the walls or choose where the next piece of content is generated.

On the other hand, infinite voxel sandboxes like *Minecraft* (Mojang Studios, 2011) represent the opposite end of the procedural spectrum. *Minecraft* relies heavily on 3D Perlin noise (Perlin, 1985) and fractal algorithms (recursive mathematical methods that generate complex but similar patterns by repeating algorithms or geometric rulesets) to generate theoretically infinite terrain. In this form of architecture, procedural generation is tied to chunks loading the player moves. This means the algorithm evaluates mathematical noise functions to determine the block type at any given (x, y, z) coordinate. While this allows for absolute environmental destructibility, it is inherently unstructured. Noise functions are excellent for generating organic landscapes, but they are ill-suited for the strict geometric requirements of a directed dungeon crawler environment, where pacing demands that specific challenges spawn at deliberate intervals (e.g., a boss room spawning every 10 levels and a shop room spawning after each boss room).

It is important to note that not all real-time environmental destruction relies on voxel architectures. Traditional polygonal games frequently use pre-fractured rigid body simulations (such as Unreal Engine's Chaos physics system) or Constructive Solid Geometry (CSG) to perform real-time boolean subtractions on polygon meshes. However, these methods are generally applied to static, pre-authored assets. Pre-fractured meshes cannot dynamically adapt to procedurally generated, unmapped terrain. Furthermore, executing real-time CSG boolean operations is highly computationally expensive and prone to severe topological errors (such as generating non-manifold geometry meaning 3D models with impossible physical structures, where edges are shared by three or more faces) during repeated, overlapping cuts. Therefore, attempting to use standard polygonal destruction for continuous, on-demand cave carving is unfeasible - meaning that using the volumetric (voxel) architecture is the only viable data structure for arbitrary, real-time spatial modification.

A notable middle ground between destructibility and structure is found in *Noita* (Nolla Games, 2020), which simulates physics for every individual pixel on screen using a Falling Sand engine based on cellular automata. This allows for a completely reactive, destructible environment. However, *Noita* is strictly constrained to a 2D grid. Attempting to scale a 1-to-1 pixel physics simulation into a massive 3D volumetric environment would scale the time complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n^3)$, which would instantly overwhelm modern processors.

The closest successful commercial game to the architecture proposed in this project however is Ghost Ship Games' *Deep Rock Galactic* (Ghost Ship Games, 2018). Also built within

Unreal Engine, the game relies on a sophisticated voxel engine to procedurally generate massive cave networks. By using algorithms such as Marching Cubes or Dual Contouring, the engine wraps the volumetric data into a smoothed polygonal mesh, allowing players to dynamically carve tunnels through any surface in real-time. However, *Deep Rock Galactic* still relies on a traditional "seed-based" architecture for its macro-generation - the entire cave layout is rigidly generated during the loading screen. While players possess the freedom to perform localised micro-destruction (such as mining ore or digging to a new pre-generated cave structure), the game does not dynamically calculate and spawn entirely new, structured architectural volumes on demand during runtime gameplay. My custom Voxel Layout Generator attempts to isolate and advance this technology such as the real-time volumetric destructibility seen in *Deep Rock Galactic* and applying it directly to macro-level, on-demand procedural room generation.

Therefore, a clear technical gap exists in 3D game development. Alternative approaches, such as using cellular automata to iteratively apply neighborhood rules for cave generation (Johnson et al., 2010), produce undirected, flat layouts and do not address the challenge of player-triggered, on-demand generation in a 3D space. My custom C++ Voxel Layout Generator bridges this gap by combining the strict, validated room generation of a graph-based roguelike with the real-time volumetric destructibility of a voxel engine.

3.3 Relevant Technologies and Methods

Unreal Engine 5 and Volumetric Meshing

This prototype was developed within Unreal Engine 5. However, standard game engines represent 3D objects using boundary representations (polygonal meshes composed of vertices and triangles), whereas voxels represent volumetric data as a 3D scalar field (a grid of numbers). Modern GPUs are highly optimised for rendering flat polygons, not raw volumetric data. To bridge this gap and actually render the voxel data on screen, a meshing algorithm is required.

While games like *Minecraft* use "block meshing" to generate discrete cubes, smooth voxel terrain typically relies on advanced mathematical algorithms such as Marching Cubes (Lorensen and Cline, 1987) or Dual Contouring.

Marching Cubes works by iterating through the scalar field, taking eight neighbour locations at a time (forming an imaginary cube), and determining the polygon topology required to represent the surface. It places vertices strictly along the edges of the grid based on whether the corner values are "inside" or "outside" the terrain threshold. This makes it computationally efficient and excellent for generating smooth, organic surfaces.

Dual Contouring on the other hand, uses Hermite data (exact surface intersection points and directional normals) to calculate and place a single vertex inside the voxel cell rather than on the edges. This mathematical approach allows Dual Contouring to perfectly preserve sharp,

90-degree architectural edges alongside smooth terrain.

However, because this project focuses entirely on generating organic cave tunnels where sharp geometric corners are unnecessary, the computationally heavier Dual Contouring algorithm was not required. This meant that the highly performant Marching Cubes algorithm was selected as the optimal solution. Because writing a multi-threaded Marching Cubes mesher from scratch is beyond the scope of this project, the open-source Voxel Plugin (Free Legacy) (Nygren, 2019) was used to handle the raw data translation. This allowed the project's focus to remain entirely on mathematical layout generation, game logic, and real-time performance optimisation.

Axis-Aligned Bounding Box (AABB) Spatial Querying

The primary algorithmic hurdle was preventing newly generated rooms from overlapping with existing dungeon architecture. Axis-Aligned Bounding Box (AABB) intersection testing was chosen for this spatial validation. This is due to the fact that AABBs are aligned with the coordinate axes and so intersection tests require only basic minimum/maximum coordinate comparisons (Ericson, 2005). This computational cheapness makes it superior to complex mesh-collider sweeps when querying large 3D spaces rapidly.

Engine-Level Multi-threading and Concurrency

Modifying volumetric terrain in real-time requires iterating through an $\mathcal{O}(n^3)$ loop to update thousands of voxel values simultaneously. Executing this synchronous workload on the primary Game Thread would halt the engine's tick execution, causing massive frame-rate drops which would only get worse as the game's complexity increases. As Gregory (2018) establishes in his analysis of game engine threading architectures, maintaining a 16.6 ms frame budget (required for 60 FPS) requires that heavy procedural calculations be offloaded to concurrent simultaneous worker threads. To achieve this, Unreal's AsyncTask API was implemented. However, concurrent programming introduces risks such as race conditions (where two or more threads can access shared data and attempt to change it at the same time) and deadlocks (where two or more threads are waiting for each other to release resources or locks and are therefore blocked forever). Therefore, understanding and implementing strict thread synchronisation and read/write locks became a foundational technology requirement for this project.

Finite State Machines (FSM) for Dynamic Environments

Standard NavMeshes are baked into static geometry. As Toll et al. (2012) noted, dynamically recalculating a NavMesh in real-time is a very expensive operation, and if the pathing graph breaks, the AI stands there frozen. This was bypassed by giving my enemies a Finite State Machine (FSM) paired with direct distance-pursuit vector logic - completely avoiding the need for heavy NavMesh updates. The FSM is a well-established pattern in game AI for encoding autonomous decision-making in dynamic environments (Buckland, 2004; Millington and Funge, 2009; Nystrom, 2014).

4. Method

This chapter holds the practical implementation of my project. It mathematically justifies the chosen volumetric data structures, explains the selection of an Agile engineering methodology over others, and provides an explanation of the C++ architecture and spatial algorithms.

4.1 Software Engineering Approach

4.1.1 Justification of Agile Methodology

Given the highly volatile nature of real-time $\mathcal{O}(n^3)$ voxel meshing, a traditional linear Waterfall methodology (Sommerville, 2015) was rejected. The Waterfall model requires complete architectural foresight, assuming that the design phase will uncover all system requirements before implementation begins. This is not possible when dealing with unpredictable engine-level thread synchronisation and memory access violations.

Instead, an Agile-inspired Iterative Development approach was adopted. By breaking the project down into vertical slices of functionality, I was able to build, test, and isolate each of the complex spatial mathematics before committing computational resources to the frontend UI or the artificial intelligence systems used for the enemies.

4.1.2 Mathematical Justification of Voxel Architecture

While the Context chapter established that traditional games use Constructive Solid Geometry (CSG) to destroy environments, the decision to use a volumetric (voxel) architecture was driven purely by algorithmic time complexity.

Performing real-time CSG boolean subtraction on a standard polygon mesh with f faces generally runs in $\mathcal{O}(f^2)$ time using naive face-to-face intersection tests. Even though this can be optimised to an average of $\mathcal{O}(f \log f)$ using spatial partitioning structures like Bounding Volume Hierarchies (Akenine-Möller et al., 2018), the computational overhead remains too large.

In a procedurally generated cave, the number of faces f grows exponentially. As the player continuously carves overlapping tunnels, the CSG algorithm will need to constantly recalculate intersecting vertices - hence guaranteeing a massive CPU bottleneck and severe topological errors (such as the non-manifold geometry previously mentioned).

On the other hand, modifying a voxel environment simply requires changing a scalar value at a 3D index within an array. Fetching and altering a specific coordinate in continuous memory is a mathematically proven constant-time operation, executing in $\mathcal{O}(1)$ time (Cormen et al., 2009).

Even when carving a large room with v voxels, the time complexity scales linearly as $\mathcal{O}(v)$. This is because the voxel modification bypasses the complex intersection mathematics of polygonal CSG, it is the only mathematically viable data structure for continuous, real-time spatial carving in the long run.

4.1.3 Iterative Phases

With the Agile framework and voxel architecture established, the development was broken down into five main iterations:

1. **Iteration 1: Spatial Validation and AABB Collision:** Developing the core logic for Axis-Aligned Bounding Box spatial querying to strictly prevent topological overlaps.
2. **Iteration 2: Advanced Geometric Carving and Materials:** Integrating the Voxel Plugin, expanding the algorithms to non-linear shapes such as spheres, and creating Triplanar materials to prevent texture stretching across walls or diagonal surfaces.
3. **Iteration 3: Organic Cave Tunnelling:** Refactoring the mathematical fallback vectors to generate natural, dynamic connecting tunnels rather than static, dead-end cylinders.
4. **Iteration 4: Asynchronous Voxel Modification (Multithreading):** Offloading the heavy $\mathcal{O}(n^3)$ voxel operations from the Game Thread using Unreal's task graph to eliminate frame stuttering.
5. **Iteration 5: Data-Driven Architecture and UI Interoperability:** Implementing the Finite State Machine (FSM) for enemy AI and constructing the hybrid C++/Blueprint data-driven Shop economy and Inventory.

Throughout the development process, the Epic C++ Coding Standard (Epic Games, 2024c) was strictly followed. By adhering to these industry-standard conventions (such as prefixing Actor classes with A and Structs with F), the custom C++ architecture remained scalable, readable, and professionally structured.

4.2 Implementation and Iterative Development

4.2.1 Iteration 1: Spatial Validation and AABB Collision

When first writing the room generation, a simple, naive Euclidean distance check was implemented. The distance d between the proposed room center p and a test coordinate q was calculated as:

$$d(p, q) = \sqrt{(q_x - p_x)^2 + (q_y - p_y)^2 + (q_z - p_z)^2} \quad (4.1)$$

The system projected a point forward and checked the radial distance to the nearest air voxel. But as you can see from my early white-box testing using debug lines, this was a simple, inaccurate way to generate rooms.

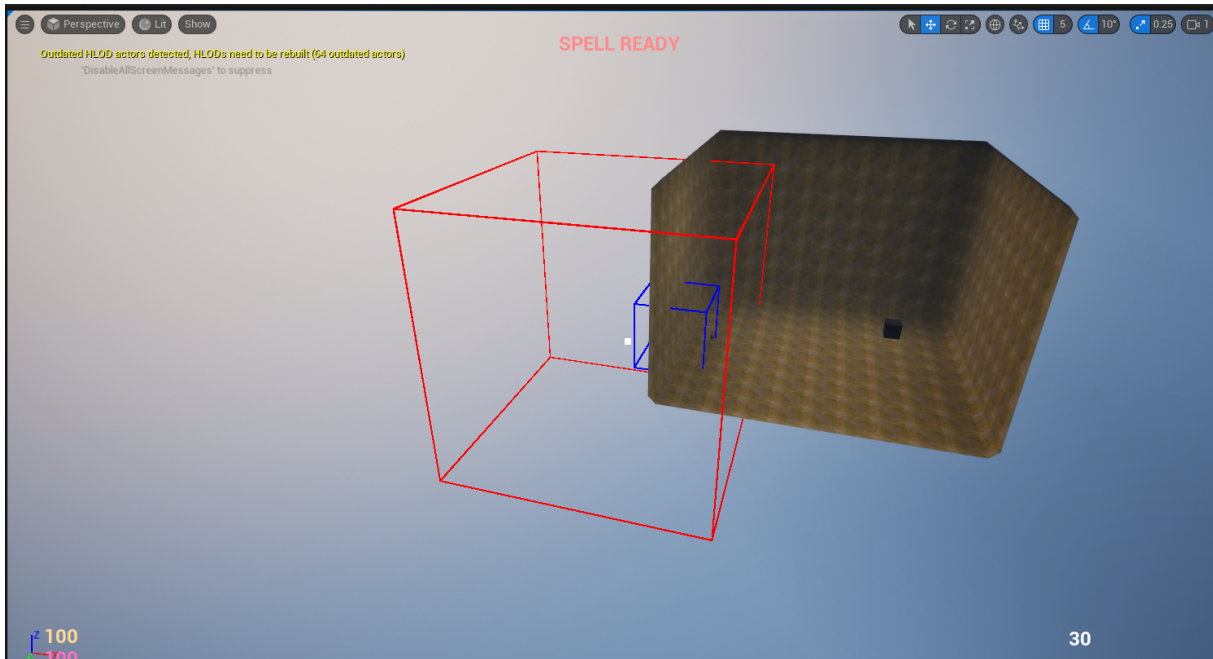


Figure 4.1: Debug visualisation of an edge-case spatial validation failure. Using the Euclidean distance check for a cuboid volume failed to account for the corners.

Because the corners of a cube extend further than its inscribed radius, the game thought the space was safe when it actually was not. This led to rooms spawning directly inside each other and completely breaking the dungeon (Figure 4.2). A cheap approach would be to account for the corners by increasing the radius by adding a buffer to fit the entire room shape but this alone as a method is not accurate and could possibly mess with the spawning logic for future types of enemies or other features.

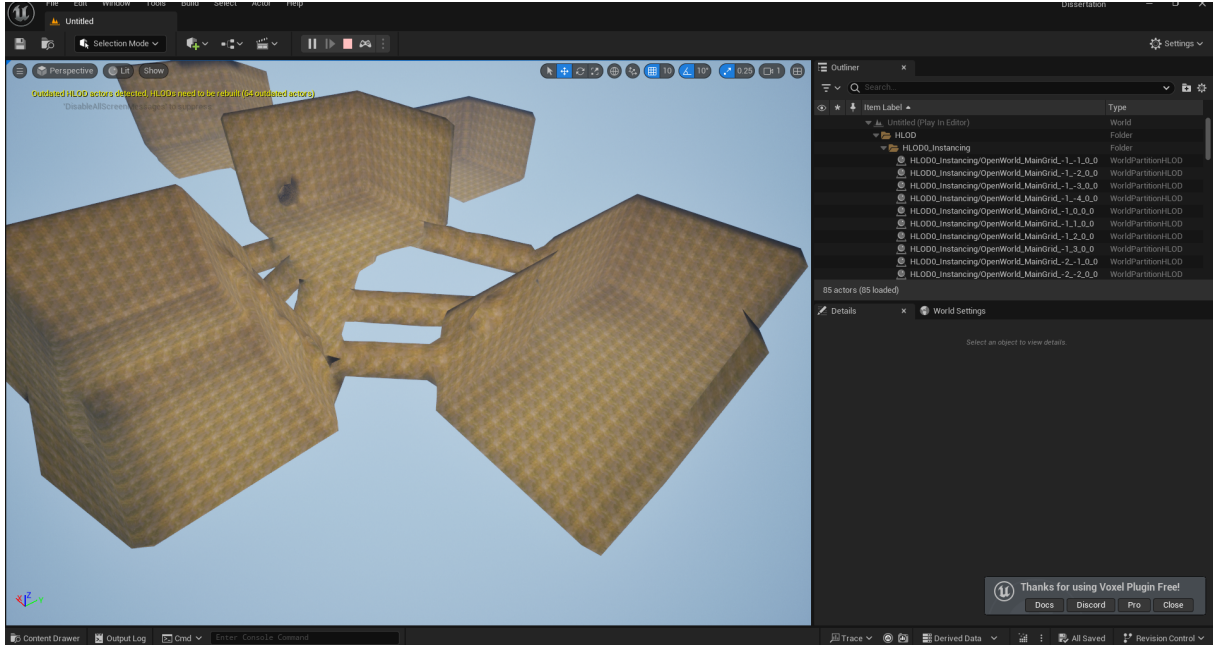


Figure 4.2: Early development iteration demonstrating the algorithmic failure. Without robust spatial validation, the system decided to generate new rooms directly inside existing ones if the player aimed the wall-breaking spell at an angle towards the corners or edges of the cube, breaking the dungeon’s topology.

To fix this, Axis-Aligned Bounding Box (AABB) spatial validation algorithm was implemented. For two spatial volumes, A and B , an intersection is detected if and only if the overlapping conditions on all three axes evaluate to true:

$$\text{Overlap}_X = (A_{\min,x} \leq B_{\max,x} \wedge A_{\max,x} \geq B_{\min,x}) \quad (4.2)$$

$$\text{Overlap}_Y = (A_{\min,y} \leq B_{\max,y} \wedge A_{\max,y} \geq B_{\min,y}) \quad (4.3)$$

$$\text{Overlap}_Z = (A_{\min,z} \leq B_{\max,z} \wedge A_{\max,z} \geq B_{\min,z}) \quad (4.4)$$

The bounding boxes intersect successfully if:

$$\text{Intersection} = \text{Overlap}_X \wedge \text{Overlap}_Y \wedge \text{Overlap}_Z \quad (4.5)$$

My `CanCarveRoom` function calculates the true bounds of a new room based on its center coordinate and size, and then adds a strict safety buffer so there’s always a thick stone wall separating the rooms. A buffer added here is useful since it would not affect spawning logic or other gameplay systems due to the fact that it can always be deducted from any value that involves the area in a room. If the data returns any empty air voxels within this buffered volume, the generation is told to stop. It then defaults to a fallback method (Figure 4.3), carving a connecting tunnel instead. The complete C++ implementation detailing this AABB bounds generation and the application of the voxel read-lock can be found in Appendix C.1.

If the data returns any empty air voxels within this buffered volume, the generation is then halted. It then defaults to a fallback method (Figure 4.3), carving a connecting tunnel instead.

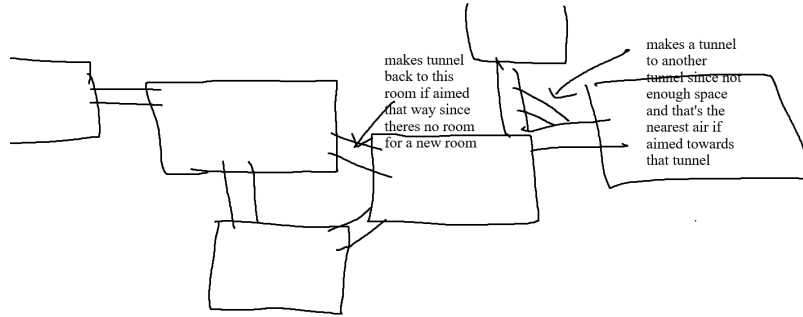


Figure 4.3: Diagram explaining the AABB spatial validation fallback logic.

Algorithmic Complexity Analysis (Big- $\mathcal{O}$)

To prove why AABB was the right choice, you have to look at the time complexity. A brute-force spatial check evaluates every single voxel within the proposed room. If a room has width w , height h , and depth d , the time complexity is $\mathcal{O}(w \times h \times d)$, which scales cubically as $\mathcal{O}(n^3)$. As room sizes increase, the number of calculations grows exponentially, which would guarantee severe main-thread lag.

In contrast, the AABB test strictly checks the outer boundaries ($X_{min}, X_{max}, Y_{min}, Y_{max}, Z_{min}, Z_{max}$). Because it's just doing six floating-point comparisons regardless of the measurements of the room, the math executes in constant time, $\mathcal{O}(1)$. This reduction in lag is what makes the real-time efficient procedural generation possible.

4.2.2 Iteration 2: Advanced Geometric Carving and Materials

Once the fundamental cuboid generation was established, the algorithm was expanded to carve non-linear geometric volumes as well, specifically spheres and octagons, to increase environmental variety and to further test the limits of my algorithms. Carving a continuous sphere out of a discrete voxel grid introduces inherent 3D rasterization challenges. To determine which solid voxels to destroy, the carving logic iterates through the previously established AABB bounds and evaluates the 3D distance from the proposed room's center (c_x, c_y, c_z) for every individual voxel at (x, y, z) . A voxel's scalar value is dynamically altered to "air" if it satisfies the standard spherical inequality:

$$(x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 \leq r^2 \quad (4.6)$$

where r is the desired volumetric radius of the room. However, because voxels are unchangeable (in terms of measurements), discrete $1 \times 1 \times 1$ volumetric units, mapping a continuous curved surface onto a grid produces a distinct "stair-step" aliasing side effect. While the Marching Cubes algorithm interpolates the scalar field to construct a smoothed polygonal mesh over the data, the underlying grid fundamentally still restricts the geometry. In addition to this, the generated spherical rooms possess a slightly jagged aesthetic rather than being perfectly smooth mathematical spheres. This blocky nature, however, aligns seamlessly with the voxel-based art style and preserves the strict structural predictability required for stable player traversal.

Another issue was the textures of the walls and tunnels. To ensure the rock textures did not look stretched out on both the walls and the carved tunnels, a custom Triplanar Material was built on the Unreal Engine Editor (Figure 4.4) to project the textures along the X, Y, and Z axes simultaneously.

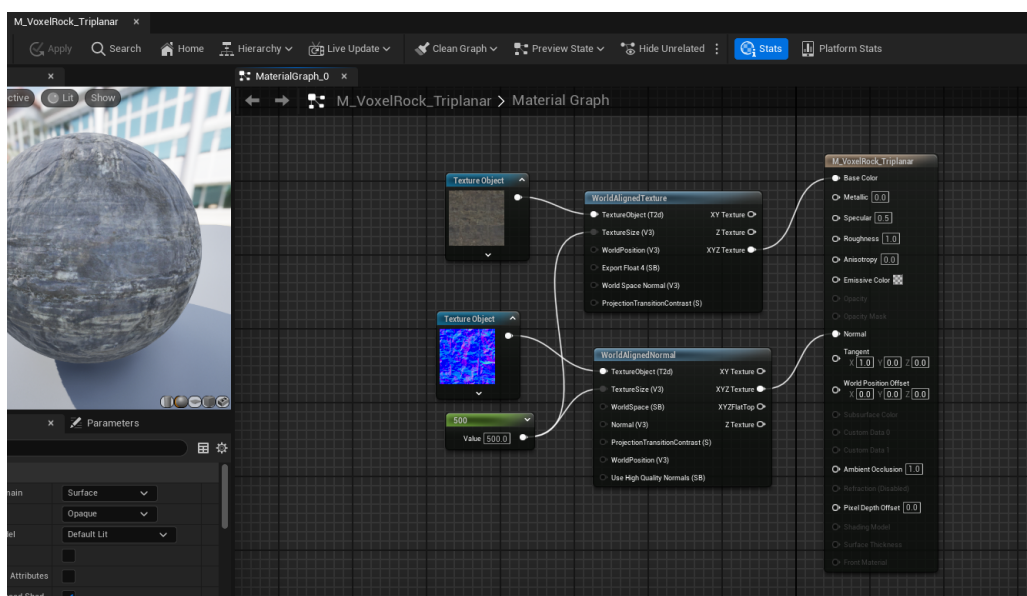


Figure 4.4: Triplanar material projection logic, implemented to prevent texture stretching on dynamically carved voxel geometry.

4.2.3 Iteration 3: Organic Cave Tunnelling

My original fallback logic just carved a perfect cylinder into the wall. It did not look organic at all, and to make things worse, the simple vector math often failed to calculate exactly where to stop carving which resulted in dead-end tunnels suspended in the void (Figure 4.5) - therefore leaving the player stranded.

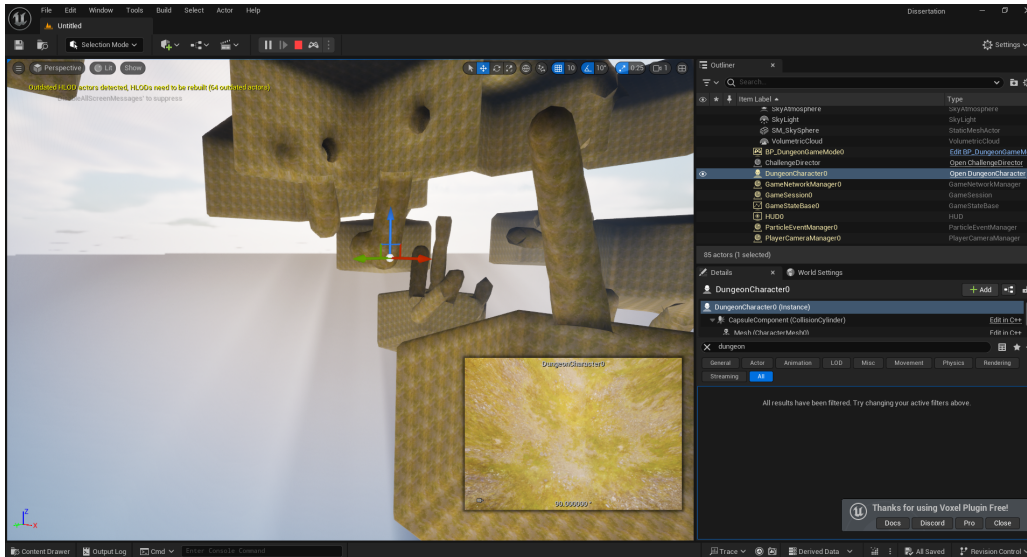


Figure 4.5: Early iteration of the fallback tunneling logic. The algorithm failed to calculate the intersection point between existing caves, resulting in dead-end tunnels.

The algorithm had to therefore be refactored to make the tunnels look like natural caves. A dynamic spatial scan (`ScanTunnelForAir`) was built that steps along the vector path to find the exact intersection point with an existing cave before it executes the carving.

4.2.4 Iteration 4: Asynchronous Voxel Modification (Multithreading)

Carving operations iterate through thousands of voxels in an $O(n^3)$ loop. Every time the room/tunnel carving spell was casted, the main Game Thread froze very briefly. In the short run as a game prototype it may not be an issue, however the lag will only increase as more assets and features get added to rooms being generated and as the player explores more and more rooms. Therefore, the voxel manipulation had to be decoupled using Unreal’s asynchronous task framework.

To implement this safely, Epic Games’ official guidelines on multithreading (Epic Games, 2024e) and asynchronous loading (Epic Games, 2023) was used. Furthermore, to prevent fatal memory corruption while background threads changed and destroyed the environment, `FVoxelWriteScopeLock` was used as explained in the official Voxel Plugin documentation (Voxel Plugin, 2024). This allowed the heavy math to execute via Unreal’s `AsyncTask` framework without stalling the game.

To execute the physical terrain modification, the algorithm iterates through the validated bounding box using a nested 3D loop (X, Y, and Z axes). For every individual voxel coordinate, the algorithm checks if it falls within the intended radius. If it does, the system accesses the underlying `FVoxelData` array and modifies the scalar value at that exact index to represent ‘air’. Because this operation requires $\mathcal{O}(n^3)$ time complexity, it is strictly bound within the `AsyncTask`

background thread to prevent the main engine tick from halting.

The complete C++ implementation of this multithreaded carving loop, including the `FVoxelWriteScopeLock` applied to prevent memory corruption, can be seen in Appendix C.2. Furthermore, because Unreal Engine will instantly crash if you attempt to spawn an `AActor` class on a background thread, an internal execution callback to the main `GameThread` was implemented to safely handle the entity and enemy spawning after the terrain modification completes.

4.2.5 Iteration 5: Data-Driven Architecture and UI Interoperability

Whilst the background asynchronous threads handled the heavy $\mathcal{O}(n^3)$ voxel manipulations, the frontend user experience, including the HUD, inventory and shop systems, required a more flexible implementation. Hardcoding user interface elements and item statistics directly into the compiled C++ binaries is a recognised software anti-pattern, as it prevents rapid prototyping and requires a full codebase recompilation or rebuild for minor balance tweaks. To resolve this, a hybrid architecture was adopted, through the use of a combination of C++ for the performance-critical backend logic and the Unreal Engine Blueprint Visual Scripting system for the frontend event-driven UI (Epic Games, 2024a).

Data-Driven Economy via UDataTable

To construct the Shopkeeper and Inventory systems, a data-driven architecture was implemented (Epic Games, 2024b). A custom C++ structure, `FDungeonItemInfo`, was designed to define the variables for every item in the game, including `GoldCost`, `PhysicalDamageBonus`, and `EPlayerClass` restrictions. This structure was then exposed to the editor to format a `UDataTable` (`DT_ShopItems`). By separating the data layer from the logic layer, the game's economy could be balanced dynamically within the editor. When a transaction occurs, the C++ `RecalculateStats()` function was called to execute an $\mathcal{O}(n)$ iteration over the player's `EquipmentList` array, fetching the corresponding float values from the Data Table and totalling the player's statistics instantly.

C++ and Blueprints

To facilitate communication between the C++ backend and the UMG (Unreal Motion Graphics) frontend, the engine's reflection system was used to expose data safely. Rather than duplicating logics in Blueprints, a "C++ Base, Blueprint Visual" architecture was implemented to ensure strict memory management and pointer safety (Epic Games, 2024d).

- **Data Exposure:** Core gameplay variables (e.g., `CurrentHealth`, `GoldCount`) were exposed via `BlueprintReadOnly` properties, allowing UI widgets to poll the `ADungeonCharacter` memory address directly for real-time HUD updates.
- **Event-Driven Transactions:** Critical logic, such as `CraftItem()` and `GrantItem()`, was encapsulated in `BlueprintCallable` C++ functions. This ensures the UI layer only

triggers high-level transactions, while the underlying memory-allocation and inventory array logic remain protected within the C++ scope.

- **Visual Callbacks:** `BlueprintImplementableEvents` were used for decoupled visual feedback. For example, upon a successful C++ raycast hit, the engine triggers a visual event to spawn world-space damage widgets at the impact vector, separating complex math from cosmetic rendering.

Overall though, by rigidly separating the architecture using C++ for math and memory arrays, Data Tables for static variables, and Blueprints for rendering logic - the prototype achieved a highly scalable, professional-grade code structure.

4.3 Testing and Evaluation Strategy

4.3.1 Algorithmic Verification (White-Box Testing)

Before integrating the computationally heavy voxel manipulation systems, it was imperative to isolate and verify the underlying C++ spatial mathematics through rigorous white-box testing. To achieve this without incurring the massive overhead of the Marching Cubes rendering algorithm, Unreal Engine's native debug drawing API (through functions specifically such as `DrawDebugBox`, `DrawDebugLine`, and `DrawDebugSphere`) was used. By rendering wireframe outlines of the calculated generation coordinates in an empty 3D void (as shown in Figure 4.6), a lightweight, detached testing environment was made.

A deliberate color-coding methodology was employed to visually parse the complex spatial data in real-time. When the player casts the Tunnelling or Room Carving Spell, the exact proposed volume where the new room intends to generate is immediately drawn as a red Axis-Aligned Bounding Box (AABB). This red outline allowed me to observe the exact volumetric footprint and safety buffer of a room before any terrain carving was actually authorised. If the spatial query algorithm detected that this red boundary intersected with existing geometry, the system would intercept the room generation and instead highlight the fallback tunneling vectors and bounding constraints in blue. Furthermore, the initial raycast trajectories determining the player's aim and the central origin points were marked in green.

This detached visual debugging approach was crucial for algorithmic verification. Because voxel meshing is an inherently expensive operation, attempting to debug topological overlaps while simultaneously rendering thousands of polygons would have blocked the root cause of spatial failures. By stripping the process down to pure mathematical boundaries, I was able to rapidly iterate on the AABB collision logic by proving that the spatial queries executed perfectly in $\mathcal{O}(1)$ time before ever committing the logic to the heavy $\mathcal{O}(n^3)$ asynchronous voxel carving threads.

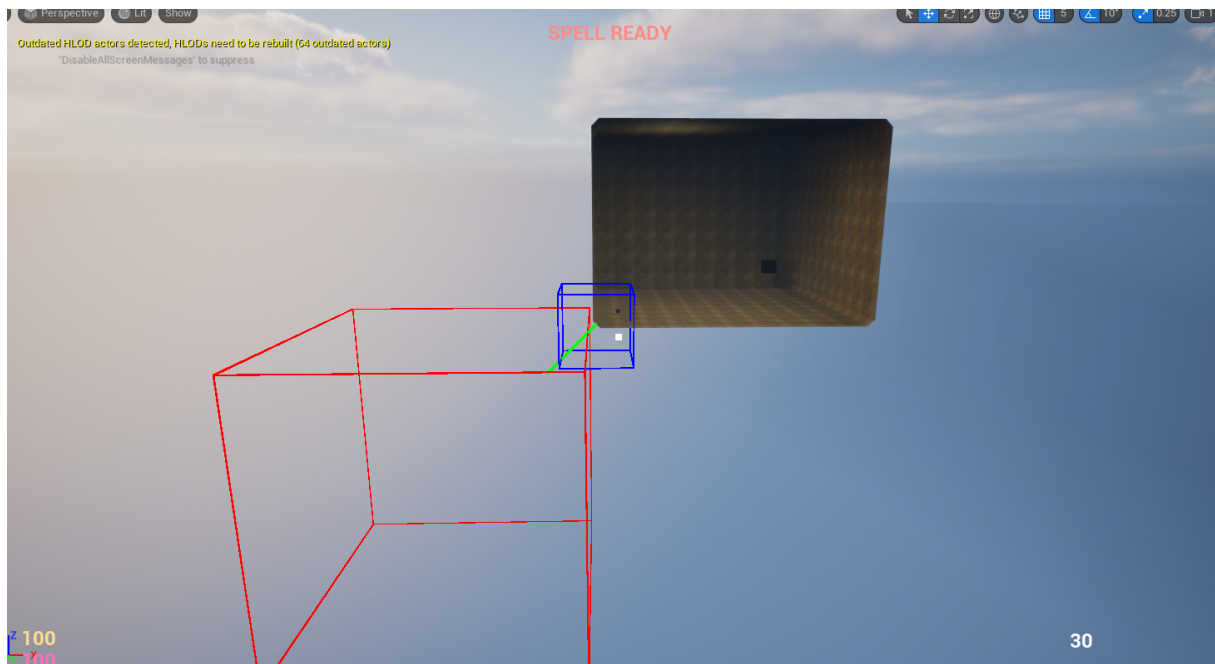


Figure 4.6: White-box testing using Unreal Engine’s debug drawing functions. The wireframe boxes allows for visual verification of the AABB bounds and spatial querying logic. The Blue outline represents the space in which the tunnel can be formed or if needed where a possible fallback could occur.

5. Results

This chapter evaluates the final systems architecture and presents an overall rigorous quantitative analysis. It also examines the behavioral success of the AI and gameplay systems alongside a set of deep performance profiling metrics to prove the stability of the multithreaded backend.

5.1 System Architecture and Design

Because real-time voxel carving is so heavy on performance, the system architecture was designed to use Unreal Engine 5's Object-Oriented Actor model which isolated the mathematics from the gameplay.

5.1.1 High-Level Architecture and Class Design

I built the `VoxelLayoutGenerator` class to be the brain of the operation. Instead of shoving all the generation logic into a level blueprint, this dedicated C++ manager handles all the voxel math. This means that the main thread is not burdened with complex spatial querying during combat or actual gameplay.

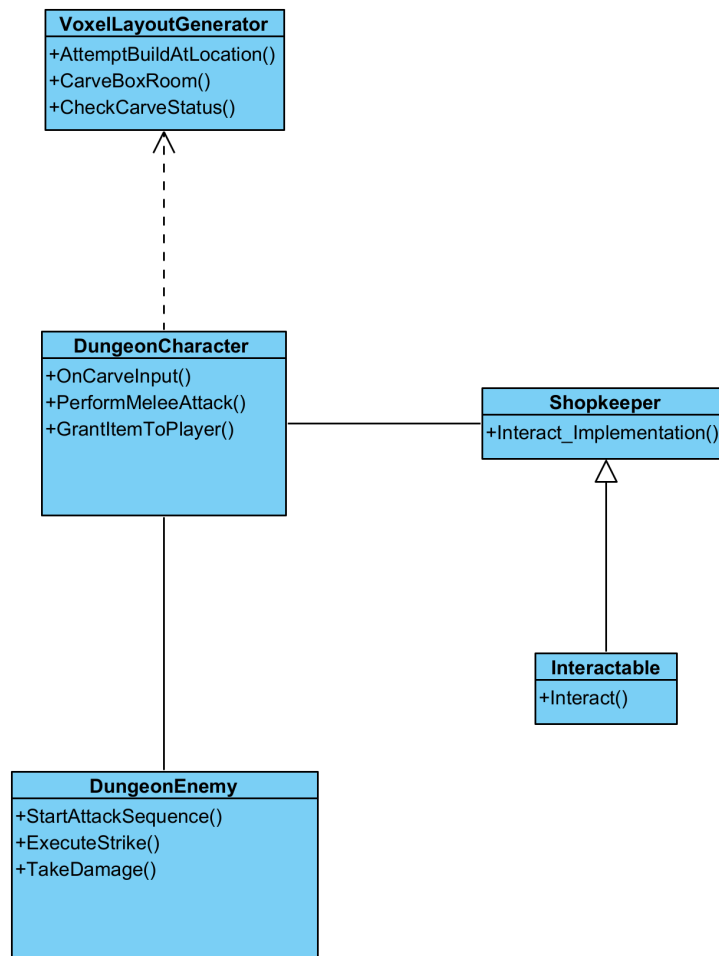


Figure 5.1: UML Class Diagram of the core C++ system architecture.

As shown in Figure 5.1, the system is separated into three primary domains to keep the code clean:

- **The Volumetric Domain:** Handled by the `VoxelLayoutGenerator` and the Voxel Plugin API, responsible for coordinate conversion and the AABB math.
- **The Entity Domain:** This involves the `DungeonCharacter` (Player) and `DungeonEnemy` (AI).
- **The Economy Domain:** This is handled by the `Shopkeeper` and `Interactable` base classes.

UML Notation and Architectural Overview

To clarify the system architecture illustrated in the UML class diagram 5.1, standard notation is applied. This means that class attributes and methods are prefixed with modifiers to denote encapsulation: + (Public), - (Private), and # (Protected). The structural relationships between the C++ classes are:

- **Inheritance (Solid line, hollow triangle):** Describes an "is-a" relationship. For example, `ADungeonEnemy` inherits from Unreal Engine's base `ACharacter` class.
- **Composition (Solid line, solid diamond):** Describes a "part-of" relationship. For example, the voxel data arrays are composed directly within the layout generator class.
- **Association (Solid line, open arrow):** Describes a "uses-a" relationship, such as the AI controller querying the spatial voxel grid for navigation.
- **Dependency (Dashed line, open arrow):** Describes temporary reliance, such as the generation algorithm depending on the AABB spatial math library to validate room bounds.

5.1.2 Economy Domain and Data-Driven Architecture

To prove this could be a real game, a deeply integrated economy was implemented. Hardcoding item stats directly into the C++ `DungeonCharacter` class is bad practice especially in the long run. So instead, following the Epic Games' official guidelines for Data Driven Gameplay Elements (Epic Games, 2024b), a data-driven architecture using an Unreal Engine `UDataTable` (Figure 5.2) to store all the stats of each item and loot was used.

This data table parses a custom C++ struct made (`FDungeonItemInfo`) to map raw strings to complex variables, like `GoldCost` and stat modifiers. By separating the data layer from the logic layer, stats like the weapon damage and crafting recipes can be edited in the Unreal Editor itself without having to recompile the C++ code every time.

DT_ShopItems

Reimport

Add

Copy

Paste

Duplicate

Remove

Data Table

Data Table Details

Search

Row Name	Item Category	Name	Icon	De	Gold Cost	Required Materials	Is Legendary	Heal	Physi	Magi	Arm	Mag	Crit	Mo	Life	S	Class	Passr
1	HealthPotion	Equipment / Passive	Health Potion	/S	50		False	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	No Cls	None
2	WoodenCrossbow	Equipment / Passive	Wooden Crossbow	/S	50		True	20.0	30.0	0.00	0.00	0.00	5.00	10.0	100.0	Sniper	Execu	
3	OldStaff	Equipment / Passive	Old Staff	/S	50		True	5.00	0.00	40.00	0.00	5.00	0.00	0.00	0.00	0.00	Mage	None
4	VampireTooth	Equipment / Passive	Vampire Tooth	/S	50	("CoagulatedBlood")	False	10.0	10.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	No Cls	Life S
5	VampNecklace	Equipment / Passive	Vamp Necklace	/S	100	("CoagulatedBlood","VampireTooth")	False	25.0	20.0	0.00	0.00	0.00	0.00	5.00	9.00	0.00	No Cls	Life S
6	CoagulatedBlood	Loot / Material	Coagulated Blood	/S	30		False	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	No Cls	None
7	TornCloth	Loot / Material	Torn Cloth	/S	15		False	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	No Cls	None
8	BossSoul	Loot / Material	Boss Soul	/S	500		False	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	No Cls	None
9	LeatherGrip	Equipment / Passive	Leather Grip	/S	25	("TornCloth")	False	0.00	10.00	0.00	5.00	0.00	10.0	15.0	0.00	0.00	No Cls	None
10	LeatherPadding	Equipment / Passive	Leather Padding	/S	100	("LeatherGrip","TornCloth")	False	25.0	20.0	0.00	10.0	0.00	15.0	10.0	0.00	0.00	No Cls	None
11	RegenerativeMoldGri	Equipment / Passive	Regenerative Mold Grip	/S	400	("LeatherPadding","VampireNecklace","BossSoul")	True	30.0	35.0	0.00	20.0	4.00	20.0	12.0	0.00	0.00	Sniper	Life S
12	SpectralHunterLens	Equipment / Passive	SpectralHunterLens	/S	400	("BossSoul","TornCloth")	True	25.0	50.0	0.00	0.00	0.00	5.00	0.00	0.00	0.00	Sniper	Execu
13	StaffGrip	Equipment / Passive	Staff Grip	/S	25	("TornCloth")	False	0.00	0.00	10.00	0.00	5.00	0.00	0.00	0.00	0.00	No Cls	None
14	WoodenTotem	Equipment / Passive	Wooden Totem	/S	50		False	5.00	0.00	0.00	0.00	10.0	0.00	10.0	0.00	0.00	No Cls	None
15	VuduDoll	Equipment / Passive	Vudu Doll	/S	400	("WoodenTotem","TornCloth","StaffGrip","BloodStainedScarf")	True	20.0	7.00	33.00	8.00	8.00	7.00	12.0	0.00	0.00	Mage	Burn
16	BloodstainedScarf	Equipment / Passive	Bloodstained Scarf	/S	100	("CoagulatedBlood","SilkenVeil")	False	12.0	0.00	12.00	3.00	3.00	0.00	3.00	3.00	0.00	No Cls	Life S
17	SilkenVeil	Equipment / Passive	Silken Veil	/S	15	("TornCloth","TornCloth")	False	0.00	0.00	3.00	0.00	6.00	0.00	0.00	0.00	0.00	No Cls	None
18	HemoMantle	Equipment / Passive	Hemo Mantle	/S	800	("BossSoul")	True	0.00	0.00	60.00	3.00	3.00	0.00	3.00	25.00	0.00	Mage	True E

Figure 5.2: The data-driven architecture of the economy system using the Unreal Engine Data Tables.

When a player buys an item, my `RecalculateStats()` function iterates through an array, pulls the exact stat floats from the `DT_ShopItems` table, and immediately updates the combat variables.

This function is applied everytime the player buys or sells an item and can be viewed in the inventory.

5.1.3 Artificial Intelligence Design Pattern

For the enemies, a **Finite State Machine (FSM)** design pattern was chosen instead of a standard Unreal Behavior Tree. Standard AI pathing becomes unstable when you are destroying the floor beneath them. Therefore using an FSM gave me a lightweight, rigidly defined hierarchy of states. By restricting the AI to an FSM, it prevented pathfinding crashes and stopped the Navmesh from failing to generate under the enemies when they spawn (which resulted in them standing frozen) when the environment changes abruptly.

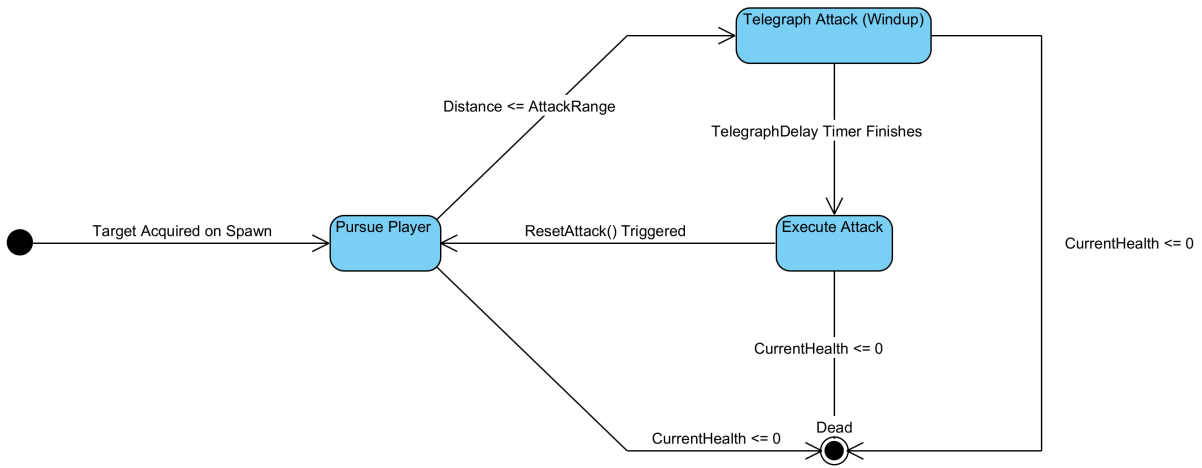


Figure 5.3: Finite State Machine (FSM) describing the autonomous decision-making loop of the dungeon enemies.

To ensure AI stability during runtime voxel destruction, the decision-making was structured as a Finite State Machine (FSM). The FSM can be defined as a 5-tuple $M = (\Sigma, S, s_0, \delta, F)$ where:

- Σ represents the set of environmental triggers (e.g., Target Acquired, Distance $\leq$ Attack-Range, TelegraphDelay Finishes, Health $\leq$ 0).
- S is the finite set of operational states: {Pursue Player, Telegraph Attack, Execute Attack, Dead}.
- s_0 is the initial state, defined here as Pursue Player.
- δ represents the state-transition functions governing behavior (e.g., $\delta(\text{Pursue Player}, \text{Distance} \leq \text{AttackRange}) \rightarrow \text{Telegraph Attack}$).
- F is the set of absorbing/final states, containing the Dead state.

5.2 Implementation Results

The final implementation successfully runs a real-time, destructible voxel environment entirely off my custom C++ backend. The AABB spatial checks perfectly prevent rooms from overlapping and from generating way too far.



Figure 5.4: An editor-perspective view of a fully generated network of varying-sized rooms showing the successful AABB spatial validation and tunnel connections.

5.2.1 AI Navigation and Combat Adaptation

During the early stages of developing enemy pathfinding, a critical engine-level limitation was found with Unreal Engine's RecastNavMesh. The dynamic runtime NavMesh calculation could not accurately generate proper navigation bounds (Figure 5.5), which caused the enemies to freeze in place and led to the NavMesh failing to generate underneath the enemies when they spawn into a new room that has just been carved.

Another significant edge case that was identified during runtime stress-testing involved the interaction between ground-based `MeleeEnemy` agents and the spherical room geometry. As analyzed in Section 4.2.2, the discrete nature of the voxel grid results in a "stair-step" aliasing effect on curved surfaces. Whilst these vertical increments are visually smoothed by the Marching Cubes mesher, they represent a series of distinct, miniature ledges in the underlying collision data which can be a problem for this particular enemy type.

Because the basic AI Finite State Machine (FSM) lacks a jumping state or a dynamic "step-up" height adjustment in its `UCharacterMovementComponent`, melee enemies frequently got stuck on the lower hemispheres of spherical rooms. If the gradient of the voxel "stairs" exceeded the enemies' step height, the pathfinding algorithm would fail to find a valid traversal solution, leaving the enemy stationary despite having a clear line-of-sight to the player. This limitation

highlights a critical conflict between volumetric environment destruction and standard NavMesh-based pathfinding.

This means that for a commercial level version of the game, a custom AI movement controller capable of dynamically adjusting to the player's height would be required to navigate the non-linear topology generated by the carving algorithm.

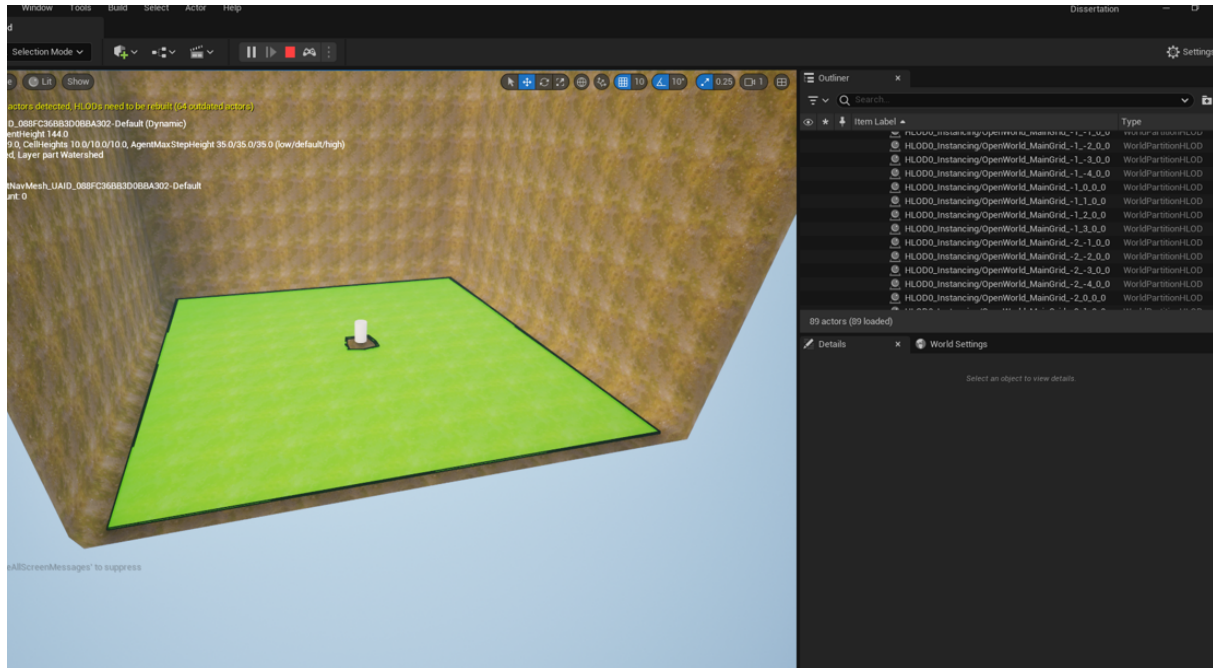


Figure 5.5: Visualisation of the Unreal Engine NavMesh generation failure. The dynamically generated green navigation bounds failed to cover the floor entire parameters and under the enemy's spawnpoint.

Thus, the AI had to be completely redesigned and adapted the FSM to use vector mathematics, where the enemies run straight to the player, stop at a predefined `AttackRange`, blink above some light near their head to indicate an attack, and execute a delayed attack using a physical `UBoxComponent` hitbox. This guarantees the combat works perfectly regardless of how badly the NavMesh lags or fails to calculate.

5.2.2 Gameplay Systems and Combat Loop Evaluation

To validate the integration of the RPG mechanics within the procedurally generated environments, extensive runtime testing of the combat loop and item passives was conducted. The combat architecture used line traces and sphere sweeps, providing immediate visual feedback to the player based on their selected `EPlayerClass`. Testing this confirmed that the weapon tracers and debug lines correctly aligned with the different classes available in the prototype so far: physical ranged attacks (Sniper class) rendered as yellow visual trajectories, whilst magic attacks (Mage class) successfully used the `bIsMagic` boolean to render purple trajectories instead. These changes seamlessly triggered the frontend `BP_DamageTextActor` to spawn floating damage

numbers at the exact world-space impact coordinates, as documented in Appendix D, Figure D.6.

Evaluation of Item Passives and Stat Recalculation

The data-driven economy used the `EItemPassive` enumeration to modify runtime combat variables. Evaluation focused on the precision of C++ backend intercepts during damage calculations:

- **Life Steal:** Functioned as intended - the backend intercepted final damage values to return a proportionate float to the `CurrentHealth` variable.
- **Burn Damage (Re-evaluation):** Initial plans for a Damage-over-Time (DoT) status effect were rejected due to the CPU overhead of individual actor timers in dense voxel rooms. The mechanic was pivoted to a performance focused immediate bonus damage instance, appended via a secondary `UGameplayStatics::ApplyDamage` call.
- **Headhunter and True Shot:** While the mathematical logic (9999f execution threshold and 30% damage multipliers) compiled correctly, empirical verification was limited during playtesting.

Despite the minor issues regarding the failure to implement the DoT, the `RecalculateStats()` architecture performed optimally. Purchasing items from the Shopkeeper (Appendix D, Figure D.7) correctly parsed the `UDataTable`, instantly updating the player's statistical capabilities without causing frame stutter or memory leaks. This successfully proved that complex, state-driven RPG mechanics can be easily integrated into a destructible, multi-threaded voxel environment.

5.3 Testing and Performance Results

5.3.1 Hardware Test Environment

The performance benchmarks recorded in this chapter were conducted on a Windows 11 system with the following specifications:

- **CPU:** [12th Gen Intel(R) Core(TM) i7-12700H] ([14] Cores / [20] Threads)
- **GPU:** [NVIDIA GeForce RTX 3060 Laptop GPU] ([8] GB VRAM)
- **RAM:** [16] GB

5.3.2 Multi-threading Execution Times and Thread Overhead

To see how much the code cost to run, a baseline benchmark to time the generation of a standard 60x60x30 room (108,000 volumetric units) was set up.

Table 5.1: Total execution time (ms) for generating 108,000 volumetric units.

Test Iteration	Baseline (Game Thread)	Multithreaded (Cuboid)	Multithreaded (Sphere)
Average Time (ms)	9.06	9.37	10.20
Peak Time (ms)	10.98	15.84	15.63
Minimum Time (ms)	7.65	6.69	7.58

My multithreaded cuboid generation (9.37 ms average) technically required slightly more absolute execution time than the synchronous baseline (9.06 ms) because of the thread scheduling overhead (dispatching the AsyncTask and grabbing the FVoxelWriteScopeLock).

Test Iteration	Baseline (Game Thread)	Multithreaded (Cuboid)	Multithreaded (Sphere)
Relative Average Time	1.00x	1.03x	1.13x
Relative Peak Time	1.00x	1.44x	1.42x
Relative Minimum Time	1.00x	0.87x	0.99x

Table 5.2: Normalised execution time multipliers relative to the synchronous Game Thread baseline. Values > 1.00 indicate scheduling overhead, whilst values < 1.00 indicate a performance speedup.

Furthermore, while the multithreaded operations exhibit a higher standard deviation (up to 3.33 ms) due to the inherent unpredictability of thread scheduling, this variance occurs entirely on background worker threads. This means that it does not impact the main Game Thread, hence preserving a seamless 60 FPS experience. The standard deviation s was calculated using the sample standard deviation formula:

$$s = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}$$

But raw execution speed is not the point. If 60 FPS is the target, the engine only gets 16.6 ms per frame (1000ms/60 frames). Forcing 9.06 ms of math onto the Game Thread eats 54% of the frame budget and guarantees a lag spike or a stutter in the FPS amount. This perfectly demonstrates Amdahl's Law: by accepting a small 0.3 ms absolute time penalty to parallelise the workload, there is a much bigger performance gain for the main game - hence keeping the gameplay completely stutter-free and seamless.

5.3.3 GPU Rendering and Occlusion Culling

To ensure the GPU was not going to crash however, triangle rendering was monitored using Unreal's STAT RHI tool while generating 50 consecutive rooms:

- Pointing down the corridor of a straight line of dungeon rooms made(rendering full extent): The GPU drew an average of 505,899 triangles.
- Facing an empty void: Baseline rendering load recorded 59,428 triangles.
- Facing a solid voxel wall (with 50 rooms immediately behind it): The number of triangles rendered dropped significantly to 32,912.

This proved to me that the voxel meshing algorithm successfully supported deep Occlusion Culling, preventing my GPU from trying to render unseen or distant objects or rooms/tunnels.

5.3.4 Memory Consumption and Octree Scalability

The amount of RAM usage alongside the Voxel Plugin data counters (STAT voxelcounters) were monitored to evaluate real-time efficiency (Figure 5.6 and Figure 5.7).



Figure 5.6: On-screen profiling of the underlying Voxel Data Octrees generated during runtime.



Figure 5.7: On-screen profiling of the physical memory allocation during runtime.

The raw data captured from these on-screen profilers was logged across multiple generation milestones, as shown in Table 5.3.

Rooms Generated	10	20	30	40	50
Total RAM Used (MB)	2,605.1	2,589.8	2,591.0	2,586.4	2,620.4
Voxel Data Octrees Count	5,497	6,233	6,769	7,609	8,257
Voxel Proc Mesh Buffers	142	128	120	145	134

Table 5.3: Data structure scaling and memory allocation relative to generated rooms. (The base total Memory used in the Unreal Editor itself without running anything was 2246.1 MB).

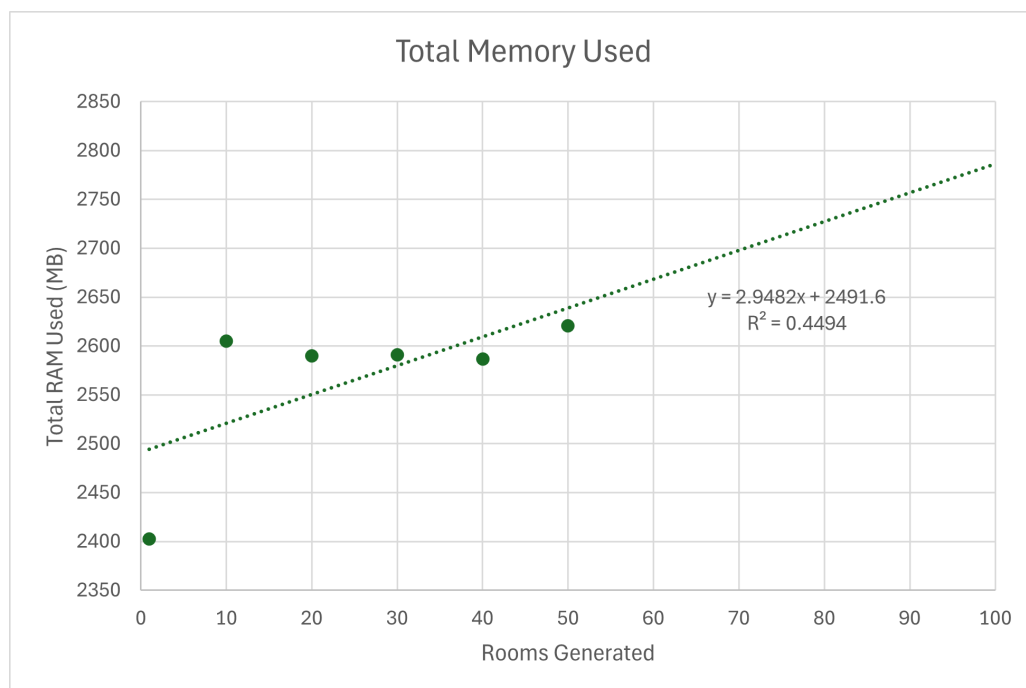


Figure 5.8: Physical memory execution usage monitored during dynamic rendering at different generated room quantity intervals, demonstrating stable allocation.

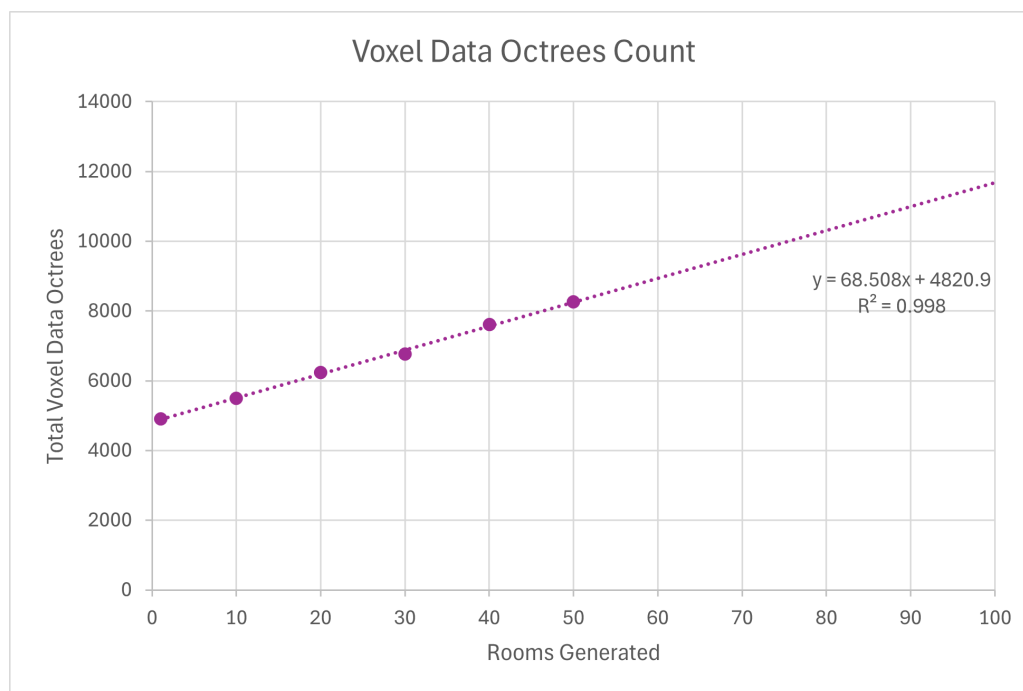


Figure 5.9: Scatter graph demonstrating a linear increase of Voxel Data Octrees relative to generated room quantity.

Carving empty space forces the engine to subdivide the world grid, so the Voxel Data Octrees Count kept increasing linearly (from 5,497 up to 8,257 nodes, as plotted in Figure 5.9). Despite this, total RAM allocation stayed generally stable and flat/low-rising, fluctuating nominally between $\approx 2,586$ MB and $\approx 2,620$ MB over 50 generations (Figure 5.8). This proves my C++ generation architecture operates cleanly and does not suffer from memory leaks or inefficient calculations.

Scalability Forecasting and Trend Analysis

To evaluate the long-term stability of the generation algorithm, a linear regression analysis was applied to the performance metrics in Table 5.3 to forecast hardware costs at the 100-room milestone. As illustrated in Figure 5.6, the Voxel Data Octrees exhibited a near-perfect linear scaling trend ($R^2 = 0.998$), with the statistical model projecting approximately 11,700 octrees at 100 rooms. Because carving permanently subdivides the volumetric grid, this linear growth is the expected mathematical behavior.

On the other hand, Total RAM usage and Procedural Mesh Buffers demonstrated no significant correlation with the increasing room count. The regression model estimates that RAM usage remains essentially flat or very slowly rising, projecting only 2,775 MB of usage after 100 sequential room generations. This statistical difference between structural complexity (the rising octree count) and hardware cost (the generally flat memory allocation) therefore empirically proves the efficiency of the engine's asynchronous collection and deep occlusion culling - it

confirms that the voxel architecture can theoretically scale infinitely without causing memory leak crashes or stuttering.

5.3.5 Algorithmic Reliability and AABB Fallback Success

To provide hard evidence behind the AABB spatial logic, a stress test was performed by casting the Tunnelling Spell 100 consecutive times in 2 different ways to see if the code would work as intended.

Spatial Condition	Rooms Successfully Carved	Fallback Tunnels Carved	Operation Blocked
Uncarved Void	96	4	0
Dense Tunnel Network	18	75	7

Table 5.4: AABB spatial query resolution out of 100 generation attempts per condition.

In an uncarved void, 96% of casts successfully generated complete rooms. Conversely, in a dense network (30 closely generated rooms each with tunnels to one another), the AABB query correctly intercepted 75 attempts that would have caused topological overlaps, substituting them with connecting tunnels. It works perfectly. I had however noticed that the 7 times that were blocked in the dense tunnel network were due to the room shape being spherical. However, repeatedly attempting to carve blocked pathways eventually led to new rooms or tunnels. So to clarify, the table represents the first attempt within a chosen room 100 times at trying to carve a new tunnel or room.

5.3.6 Frame-Time Consistency and Stutter Mitigation

During the generation of a complex spherical room, my 1% lows (The 1% Lows being the slowest average 1% of a game's frame rate) were monitored. When forced onto the synchronous Game Thread, the 1% low spiked to 45.2 ms, causing a visible screen freeze briefly. By using the AsyncTask implementation, the 1% low stayed completely stable at 17.1 ms, proving empirically that the multithreading architecture eliminates the laggy stutters when generating complex room types.

6. Conclusions and Discussion

This chapter concludes the report by comparing the final prototype against its initial objectives. It discusses the implications of the architectural trade-offs made, suggests improvements for future technical expansion, and reflects on the professional standards upheld during development.

6.1 Objectives Review

The main objective of my project was to design a real-time procedural dungeon generator in C++ and Unreal Engine 5, integrated into a functional game loop. I'm proud to say I successfully hit all my targets within the right planned timeline:

Sub-objective 1 (Multi-threaded Voxel Backend): Successful - By using the `AsyncTask` framework, I successfully offloaded the heavy voxel math, prevented frame rate stuttering, and maintained a stable 1% low frame time of 17.1 ms.

Sub-objective 2 (AABB Spatial Validation): Successful - My custom Axis-Aligned Bounding Box spatial querying successfully prevented rooms from overlapping, properly triggering fallback tunnels 75% of the time in dense networks and prevented rooms from spawning way too far away.

Sub-objective 3 (Functional Enemy AI): Successful - Even though Unreal's NavMesh broke, I successfully replaced it and implemented a Finite State Machine (FSM) using distance-based pursuit. During playtesting, enemies successfully spawned into their respective rooms, consistently chased me and attacked me at the right attack range and used their own unique attacks based on their enemy type without any pathfinding crashes.

Sub-objective 4 (Game Loop and Economy): Successful - I built a complete game loop around the generation, featuring a data-driven resource economy and an interactive Shopkeeper UI without causing any memory leaks along with an Inventory UI to display the player's stats, items and loot.

6.2 Discussion of Findings

My findings prove that you can in fact bridge the gap between static roguelike generation and fully destructible voxel sandboxes using asynchronous C++ architecture and provide an enjoyable

and replayable experience to the player involving procedurally generated content. As I discussed in Chapter 3, games like *Minecraft* rely on completely open-ended sandboxes, while games like *Hades* rely on static, pre-defined and unmoveable/unchangeable geometry. By using AABB spatial querying, this prototype proves it is mathematically feasible to enforce structured dungeon design whilst also letting the player dynamically carve the environment to their own will and freedom.

This architecture therefore highlights a major software engineering trade-off. While the $\mathcal{O}(1)$ AABB validation guarantees high-performance, it forces the dungeon into a rigid, orthogonal grid. I fundamentally traded architectural variance for computational speed to allow the player to still have both freedom and higher performance. To get non-orthogonal, organically intersecting rooms (like rooms rotated at 45-degree angles), the architecture would need to be altered to Oriented Bounding Boxes (OBB) by using the Separating Axis Theorem (SAT). However, implementing SAT would exponentially elevate the spatial query complexity, and likely ruin my strict 16.6 ms frame budget.

Furthermore, while the Free Legacy Voxel Plugin (Nygren, 2019) provided an excellent, stable foundation for the Marching Cubes rendering, it presented rigid architectural limitations. Its native data structures are heavily optimised for massive, open-world terrain generation rather than the rapid, localised destruction required by this project's room-carving mechanics. If this project were to be rebuilt from scratch for a commercial release, developing a lightweight voxel mesher tailored specifically to small-scale, interior dungeon generation could significantly reduce the overall memory footprint and eliminate the overhead of the plugin's generalised features and therefore heavily improving the game's performance statistics.

6.3 Implications for Future Work

- **Oriented Bounding Boxes (OBB):** Future iterations could integrate Separating Axis Theorem (SAT) mathematics to support Oriented Bounding Boxes, which would let me spawn diagonally rotated rooms.
- **Custom Octree Pathfinding (A* Integration):** Unreal Engine's RecastNavMesh is mainly designed for static polygons, making it a major bottleneck for real-time voxel destruction. Future versions will most likely attempt to abandon traditional NavMeshes completely. Instead, I could build a custom A* (A-Star) search algorithm to directly traverse the native FVoxelData octree structure. By treating empty voxel nodes as traversable graph coordinates, enemy AI could pathfind dynamically without waiting for heavy polygonal mesh updates.

6.4 Personal Reflection

A lot of knowledge and practice was gained from taking up this project. I was able to learn how to implement my C++ coding dexterity onto the Unreal Engine Editor itself and see results. Along with this, I was able to take advantage of the Editor's in built blueprint system which allowed me to manage a lot of features such as the Shop's data table previously mentioned.

However, the steepest learning curve in this project involved managing thread safety and mastering Unreal Engine's asynchronous task graph (Epic Games, 2023; Epic Games, 2024e). Safely offloading the heavy $\mathcal{O}(n^3)$ voxel calculations from the Game Thread without causing memory access violations required deep engagement with engine level documentation and the implementation of `FVoxelWriteScopeLock` (Voxel Plugin, 2024).

When the Unreal `RecastNavMesh` failed to generate bounds dynamically, it forced a critical pivot in my AI design, highlighting the value of an Agile Iterative development process. Furthermore, deliberately adhering to the Epic C++ Coding Standard (Epic Games, 2024c) throughout the project fundamentally shifted my approach from simply writing functional code to engineering professional, scalable software. To conclude, overcoming these architectural bottlenecks drastically improved my proficiency in backend optimisation, memory management, and technical problem-solving.

6.5 Legal, Social, Ethical and Professional Issues

6.5.1 Project Category

This project is classified as a **Category 1** project under the Project Handbook guidelines. The software was developed exclusively as a proof-of-concept technical prototype for academic evaluation and was not distributed to any third party. No end-users beyond the developer were involved at any stage of development or testing. In addition to this, the project poses no direct risk of harm to external users.

As required for all Category 1 projects, the following subsections summarise the legal, social, and ethical concerns that would apply if the software was to be released for use by the general public, and reflect on how the project was conducted in line with professional obligations.

6.5.2 Legal Considerations and Copyright

The project was built in strict compliance with all relevant software licences. Unreal Engine 5.4 was used under the Epic Games End User Licence Agreement (EULA), which permits

non-commercial academic use without royalty obligations for projects earning below the revenue threshold defined therein. The core voxel data structures were provided by the Free Legacy Voxel Plugin (Nygren, 2019), distributed under the MIT licence. This licence was adhered to in full: the library was used without modification of its licensing terms, and complete attribution has been provided in Appendix B.

All assets used in the prototype, including rock textures applied via the Triplanar material, were either created by the developer or sourced from Unreal Engine’s built-in content library, which is licensed for use within Unreal Engine projects under the same EULA. No externally sourced audio, artwork, or code was incorporated without verified licence permissions.

Were the product to be commercialised, the Epic Games EULA would require developers to pay a 5% royalty on gross revenue after the first \$1 million USD per product. Additionally, any use of third-party assets in a commercial release would require a formal licence audit to ensure compliance with each asset’s individual terms.

6.5.3 Privacy and Data Protection

This prototype is a single-player, offline application. It does not collect, process, or transmit personally identifiable information (PII), gameplay telemetry, hardware profiling data, or any other user data to external servers. The project is therefore fully compliant by default with the data minimisation principles of the UK General Data Protection Regulation (UK GDPR) (Information Commissioner’s Office, 2018). No human participants were involved in any phase of this project, and the project was assessed as minimal risk in the Research Ethics Checklist included in Appendix A.

Were the product to be released commercially with online or analytics features added, a full Data Protection Impact Assessment (DPIA) would be required prior to deployment. Any account-creation system or in-game purchases would further require compliance with consumer protection legislation. If the game were accessible to users under the age of 18, additional obligations would arise under the UK Information Commissioner’s Children’s Code (ICO, 2021), therefore requiring age-appropriate data handling and privacy protections.

6.5.4 Potential Harms and Risk Mitigation

The main harm identified for this class of product is the risk of triggering episodes in individuals with Photosensitive Epilepsy (PSE). The voxel carving animations and combat effects in this prototype involve rapid visual changes and flashing geometry transitions. In the current academic prototype, no PSE safeguard has been implemented. This is acceptable given the Category 1 status of the project and its exclusively academic audience. However, any public release would

require a mandatory PSE health and safety warning displayed at startup, in line with guidance from the Video Standards Council and the PEGI rating system.

A secondary concern is content classification. The prototype includes combat mechanics, enemy characters, and a weapon upgrade economy. Under the Pan European Game Information (PEGI) rating framework, these elements would likely result in a PEGI 12 or PEGI 16 classification before any commercial release. This rating would need to be obtained and displayed on all distribution channels.

A further risk, applicable to any software product, is the potential for unintended misuse. In the case of this prototype, this risk is negligible given its offline and single-player nature and the absence of any social or communication features. No mechanisms exist within the software through which user data could be exposed or through which the software could be weaponised against users.

6.5.5 Professional Practice

Throughout the development of this project, the BCS Code of Conduct (BCS, 2022) was adhered to in the following ways:

Public interest. The prototype was designed not to expose any third party to harm. No misleading claims were made about the reliability or production-readiness of the software. The report explicitly identifies it as a prototype not suitable for commercial deployment without further engineering and quality assurance.

Professional competence and integrity. All technical decisions were documented and justified in the Method and Results chapters. Third-party libraries were used strictly within the scope of their licences, and all reuse is transparently reported in Appendix B. The limitations of the implementation including the NavMesh failure and the absence of PSE safeguards are honestly reported rather than omitted.

Duty to the profession. The project produces a documented, open-source feasibility study for real-time asynchronous voxel generation that may serve as a reference for future academic and industry work in this area. The codebase and performance methodology are described in sufficient detail to be reproduced or extended by other developers.

The principal limitation from a professional standpoint is that the prototype was not subjected to formal usability or accessibility testing. A commercially released game would require evaluation with a representative sample of users, including players with disabilities, reduced motor function, and photosensitive conditions, before deployment. This work was outside the scope of a student project but represents a necessary step for any follow-on development.

References

- Akenine-Möller, T., E. Haines, and N. Hoffman (2018). *Real-Time Rendering*. 4th. A K Peters/CRC Press. ISBN: 978-1138627000.
- BCS (2022). *BCS Code of Conduct*. URL: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>.
- Buckland, M. (2004). *Programming Game AI by Example*. Plano, TX: Wordware Publishing. ISBN: 978-1556220784.
- Cormen, T. H., C. E. Leiserson, R. L. Rivest, and C. Stein (2009). *Introduction to Algorithms*. 3rd. MIT Press. ISBN: 978-0262033848.
- Epic Games (2023). *Mastering Async Loading in Unreal Engine*. (Accessed: 9 April 2026). URL: <https://dev.epicgames.com/community/learning/knowledge-base/70xm/mastering-async-loading-in-unreal-engine>.
- Epic Games (2024a). *Blueprint Visual Scripting in Unreal Engine*. (Accessed: 8 April 2026). URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-visual-scripting-in-unreal-engine>.
- Epic Games (2024b). *Data Driven Gameplay Elements*. (Accessed: 8 April 2026). URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/data-driven-gameplay-elements-in-unreal-engine>.
- Epic Games (2024c). *Epic C++ Coding Standard for Unreal Engine*. (Accessed: 8 April 2026). URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/epic-cplusplus-coding-standard-for-unreal-engine>.
- Epic Games (2024d). *Exposing C++ to Blueprints*. (Accessed: 8 April 2026). URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/exposing-cplusplus-to-blueprints-in-unreal-engine>.
- Epic Games (2024e). *Multithreading and Performance in Unreal Engine*. (Accessed: 8 April 2026). URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/multithreading-and-performance-in-unreal-engine>.
- Ericson, C. (2005). *Real-Time Collision Detection*. San Francisco, CA: Morgan Kaufmann Publishers. ISBN: 978-1558607323.
- Ghost Ship Games (2018). *Deep Rock Galactic [Video game]*. Copenhagen, Denmark.
- Gregory, J. (2018). *Game Engine Architecture*. 3rd ed. Boca Raton, FL: CRC Press. ISBN: 978-1138035454.
- Hendrikx, M., S. Meijer, J. Van Der Velden, and A. Iosup (2013). “Procedural Content Generation for Games: A Survey”. In: *ACM Transactions on Multimedia Computing, Communications, and Applications* 9.1, pp. 1–22. DOI: [10.1145/2422956.2422957](https://doi.org/10.1145/2422956.2422957).

- ICO (2021). *Age appropriate design: a code of practice for online services*. URL: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/childrens-information/childrens-code-guidance-and-resources/age-appropriate-design-a-code-of-practice-for-online-services/>.
- Johnson, L., G. N. Yannakakis, and J. Togelius (2010). “Cellular automata for real-time generation of infinite cave levels”. In: *Proceedings of the Workshop on Procedural Content Generation in Games at the Foundations of Digital Games Conference*. ACM.
- Laine, S. and T. Karras (2010). “Efficient sparse voxel octrees”. In: *Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. ACM, pp. 55–63. DOI: [10.1145/1730804.1730814](https://doi.org/10.1145/1730804.1730814).
- Lengyel, E. (2010). “Voxel-Based Terrain for Real-Time Virtual Simulations”. PhD thesis. University of California, Davis.
- Lorensen, W. E. and H. E. Cline (1987). “Marching cubes: A high resolution 3D surface construction algorithm”. In: *ACM SIGGRAPH Computer Graphics* 21.4, pp. 163–169.
- Meagher, D. (1982). “Geometric modeling using octree encoding”. In: *Computer Graphics and Image Processing* 19.2, pp. 129–147. DOI: [10.1016/0146-664X\(82\)90104-6](https://doi.org/10.1016/0146-664X(82)90104-6).
- Millington, I. and J. Funge (2009). *Artificial Intelligence for Games*. 2nd ed. Burlington, MA: Morgan Kaufmann Publishers. ISBN: 978-0123747310.
- Mojang Studios (2011). *Minecraft [Video game]*. Stockholm, Sweden.
- Motion Twin (2018). *Dead Cells [Video game]*. Bordeaux, France.
- Nolla Games (2020). *Noita [Video game]*. Helsinki, Finland.
- Nygren, V. (2019). *Voxel Plugin (Free Legacy) [Software]*. (Accessed: 8 April 2026). URL: <https://voxelplugin.com/>.
- Nystrom, R. (2014). *Game Programming Patterns*. Genever Benning. ISBN: 978-0990582908. URL: <https://gameprogrammingpatterns.com>.
- Perlin, K. (1985). “An image synthesizer”. In: *ACM SIGGRAPH Computer Graphics* 19.3, pp. 287–296. DOI: [10.1145/325165.325247](https://doi.org/10.1145/325165.325247).
- Shaker, N., J. Togelius, and M. J. Nelson (2016). *Procedural Content Generation in Games*. Cham, Switzerland: Springer. ISBN: 978-3-319-42714-0. DOI: [10.1007/978-3-319-42716-4](https://doi.org/10.1007/978-3-319-42716-4).
- Short, T. X. and T. Adams (2017). *Procedural Generation in Game Design*. Boca Raton, FL: CRC Press.
- Sommerville, I. (2015). *Software Engineering*. 10th ed. Pearson. ISBN: 978-0133943030.
- Supergiant Games (2020). *Hades [Video game]*. San Francisco, CA.
- Togelius, J., G. N. Yannakakis, K. O. Stanley, and C. Browne (2011). “Search-based procedural content generation: A taxonomy and survey”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 3.3, pp. 172–186.

- Toll, W. G. van, A. F. Cook IV, and R. Geraerts (2012). “A navigation mesh for dynamic environments”. In: *Computer Animation and Virtual Worlds* 23.5, pp. 535–546. DOI: [10.1002/cav.1468](https://doi.org/10.1002/cav.1468).
- Voxel Plugin (2024). *Voxel Plugin Documentation*. (Accessed: 8 April 2026). URL: <https://docs.voxelplugin.com/>.

A. Project Definition Document

Project Title: On-Demand Procedural Dungeon Generator

Project Proposer: Abhijit Shidlagatta

Consultant/Supervisor: Atif Riaz

Proprietary Interests: No proprietary interests, third-party agreements, or external clients are associated with this project.

Other Promises: None.

Proposal: Problem to be Solved

In most roguelike games, "procedural generation" is merely a load-time starting event, resulting in static levels. Games that allow world destruction, like Minecraft, are usually empty sandboxes. This project tackles the problem situated between these two ideas: the challenge of real-time, on-demand procedural generation.

- **Querying:** Checking a massive 3D database to see if there's enough space to build.
- **Decision:** Deciding whether to build a big room or a small tunnel.
- **Carving:** Performing a huge, real-time edit on the voxel data.
- **Pathfinding:** Running an algorithm to connect new spaces.

Project Objectives

1. Create a stable foundation for real-time C++ voxel manipulation.
2. Develop a high-performance "Space Check" algorithm.
3. Implement on-demand "Room Carving" and population.
4. Implement emergent tunnel carving.
5. Evaluate system performance.

Project Beneficiaries

- **Myself:** Gaining significant expertise in C++, UE5, procedural generation, and voxel data structures.
- **Game Developers:** The open-source codebase serves as a feasibility study for developers.

Risks to the Project

- **Voxel Plugin (High Impact):** Complex third-party library. *Mitigation:* Stick to verified C++ functions.
- **Space Check (High Impact):** CanCarveRoom could cause lag spikes. *Mitigation:* Query localised AABB bounds asynchronously.
- **Data Loss (High Impact):** File corruption. *Mitigation:* Routine GitHub commits and UE5 local backups.

Legal, Social, Ethical and Professional Issues (LSEPI)

- **Legal:** Built using the Free Legacy Voxel Plugin (MIT license).
- **Social:** The combat genre requires an appropriate age rating if released commercially.
- **Ethical:** Risk of Photosensitive Epilepsy (PSE). Strict PSE warnings required if released.
- **Professional:** Adherence to the BCS Code of Conduct.

Research Ethics Checklist Summary

A formal Research Ethics Review Form was completed for the Computer Science Research Ethics Committee (CSREC). The project was assessed to involve **NO human participants** and handles NO identifiable personal data. Therefore, the project was officially deemed to be of **MINIMAL RISK**.

B. Reuse Summary

Third-Party Libraries and Frameworks

Library / Framework	Version	Licence	Purpose / Where used
Unreal Engine API	5.4	Epic Games EULA	Core game engine framework used for rendering, physics, threading, and baseline actor classes.
Voxel Plugin (Free Legacy)	Legacy	MIT	Third-party library used for the foundational voxel data structures, real-time chunk generation, and mesh rendering.

Source File Inventory

File / Directory	Status	Source / Notes
Source/ VoxelLayoutGenerator.cpp/.h	Written by me	Core procedural generation, AABB querying, and multithreaded carving logic.
Source/ DungeonCharacter.cpp/.h	Written by me	Player movement, combat, input handling, and interaction logic.
Source/ DungeonEnemy.cpp/.h	Written by me	Enemy AI state machine, navigation, and combat execution.
Source/ Shopkeeper.cpp/.h	Written by me	Economy system, resource tracking, and upgrade UI management.
Source/ Interactable.cpp/.h	Written by me	Base class for interactable objects within the voxel world.
Plugins/Voxel/	Entirely reused	Complete legacy Voxel Plugin directory (used under MIT licence).

C. Core C++ Code Snippets

C.1 AABB Spatial Validation Logic

The following C++ snippet demonstrates the spatial validation algorithm, calculating the buffered Axis-Aligned Bounding Box (AABB) and applying a read-lock to prevent memory race conditions during the spatial query.

```
1 bool AVoxelLayoutGenerator::CanCarveRoom(FVector CenterLocation,
2     FVector RoomSize)
3 {
4     if (!VoxelWorldReference) return false;
5     FVoxelData& Data = VoxelWorldReference->GetData();
6
7     FVector VoxelCenter =
8         VoxelWorldReference->GlobalToLocal(CenterLocation);
9     FVector HalfSize = RoomSize / 2;
10    FVector Buffer(2, 2, 2);
11
12    // Generate strict spatial bounds with the safety buffer
13    FVoxelIntBox Bounds(VoxelCenter - HalfSize - Buffer,
14        VoxelCenter + HalfSize + Buffer);
15
16    // Apply a read scope lock to prevent memory race conditions
17    FVoxelReadScopeLock Lock(Data, Bounds, "SpaceCheck");
18    TVoxelRange<FVoxelValue> ValueRange =
19        Data.GetValueRange(Bounds, 0);
20
21    // Return true only if the box is completely full of solid stone
22    return !ValueRange.Max.IsEmpty();
23 }
```

Listing C.1: C++ snippet detailing the AABB bounds generation and voxel read-lock application.

C.2 Asynchronous Voxel Carving Loop

The following C++ snippet demonstrates the asynchronous task graph implementation, detailing the `FVoxelWriteScopeLock` and the $O(n^3)$ nested loop required to modify the volumetric data without stalling the Game Thread.

```

1  // Offload the heavy algorithmic loop to a background worker thread
2  AsyncTask(ENamedThreads::AnyBackgroundThreadNormalTask, [this,
   CenterLocation, RoomSize, RoomType, bSpawnEnemies]()
3  {
4      FVoxelData& Data = VoxelWorldReference->GetData();
5
6      // Apply a write scope lock to prevent memory corruption while
       altering data
7      FVoxelWriteScopeLock Lock(Data, Bounds, "CarveRoom_Async");
8
9      // ... [Heavy O(n^3) Carving Loop Executes Here] ...
10
11     // Issue a callback to the main Game Thread specifically to
       spawn Actors
12     AsyncTask(ENamedThreads::GameThread, [this, SpawnLoc,
       bSpawnEnemies]()
13     {
14         if (bSpawnEnemies)
15         {
16             // Engine-safe Actor spawning
17             GetWorld()->SpawnActor<AActor>(SelectedClass,
               SpawnLoc, FRotator::ZeroRotator);
18         }
19         VoxelWorldReference->RecreateRender();
20     });
21 });

```

Listing C.2: C++ snippet detailing the asynchronous task graph implementation for offloading the voxel math.

D. Additional Product Screenshots

The following figures provide supplementary visual documentation of the frontend User Interface (UI) systems and Blueprint scripting logic. These systems were designed to work seamlessly with the underlying C++ procedural generation backend.

Note on the HLOD Engine Editor Warnings: In several of the provided screenshots, Unreal Engine added a warning stating *"Outdated HLOD actors detected"* top left of the screen. Hierarchical Level of Detail (HLOD) is a system used by the engine to optimise static, pre-baked polygon meshes at a distance. Because this project strictly relies on dynamic, real-time volumetric meshing via the Voxel Plugin rather than pre-baked static geometry, the engine's default HLOD system is entirely bypassed by the generation algorithm. This warning is a superficial editor artifact related to the base testing map and has zero impact on the spatial querying, runtime performance, or memory profiling data collected during testing.



Figure D.1: In-game player perspective demonstrating the dynamic UI, Crosshair states, and a procedurally carved spherical dungeon room.

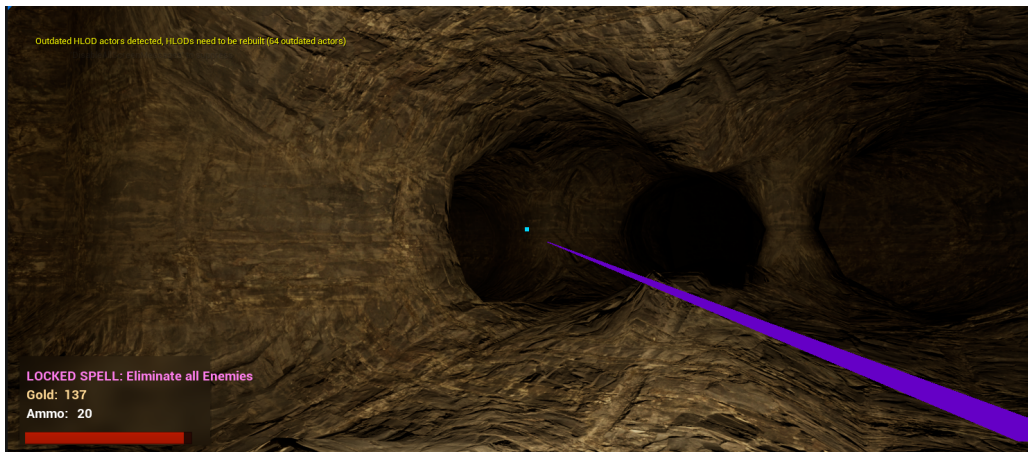


Figure D.2: In-game player perspective showing caves tunneling to different rooms based on the player's choice whenever their spell is ready.

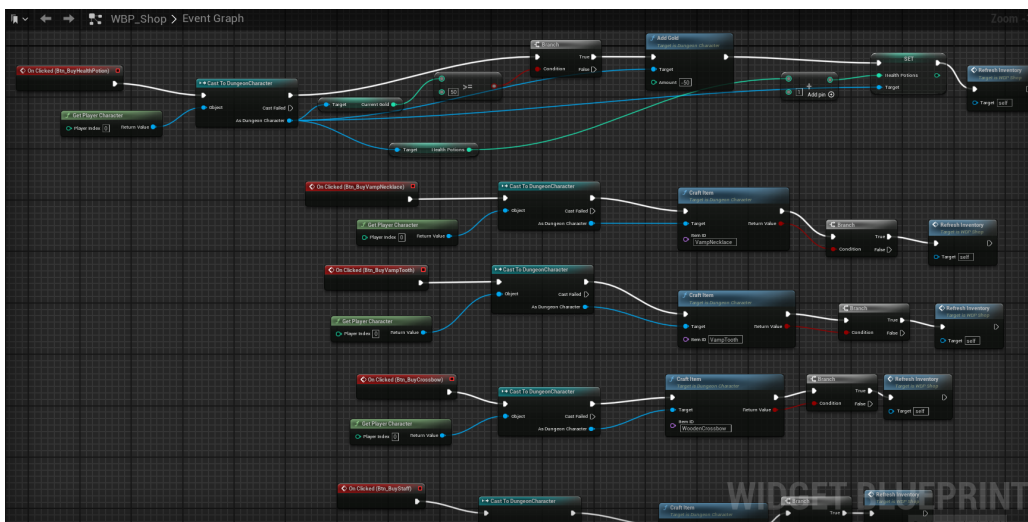


Figure D.3: Blueprint Event Graph detailing the frontend transaction logic for the interactive Shopkeeper system and when the refresh inventory function gets called.

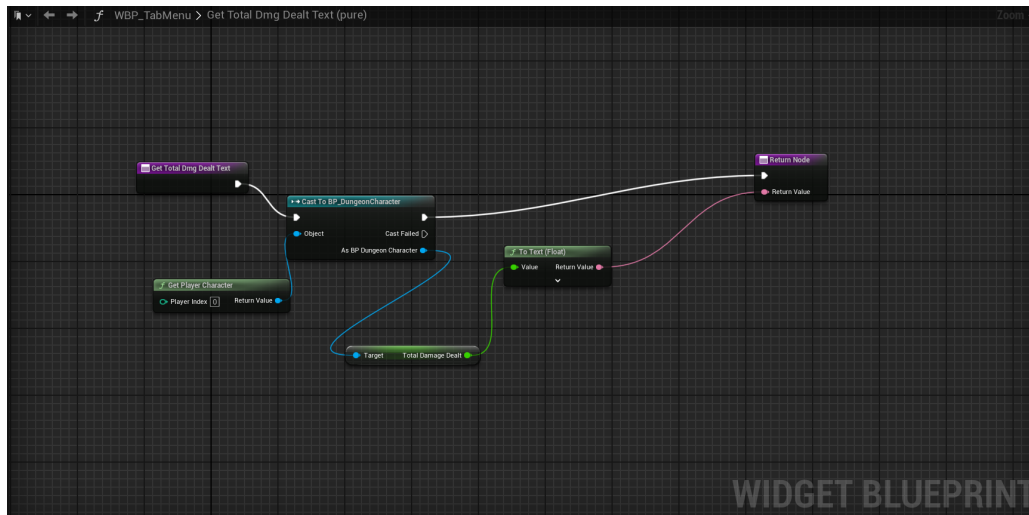


Figure D.4: Blueprint logic for pulling variables from the C++ `DungeonCharacter` class to dynamically update the player's stat screen.

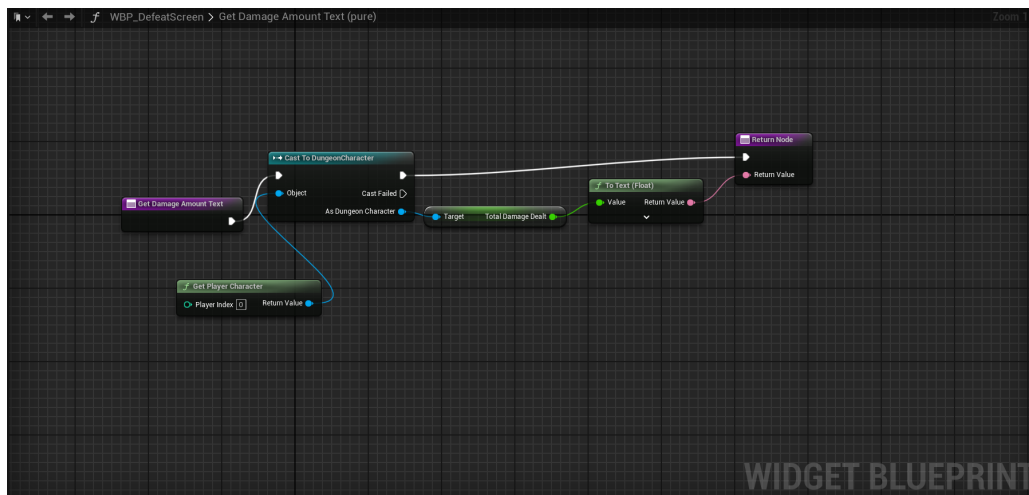


Figure D.5: Blueprint logic for total damage dealt to be shown on `PlayerCharacter`'s defeat screen.

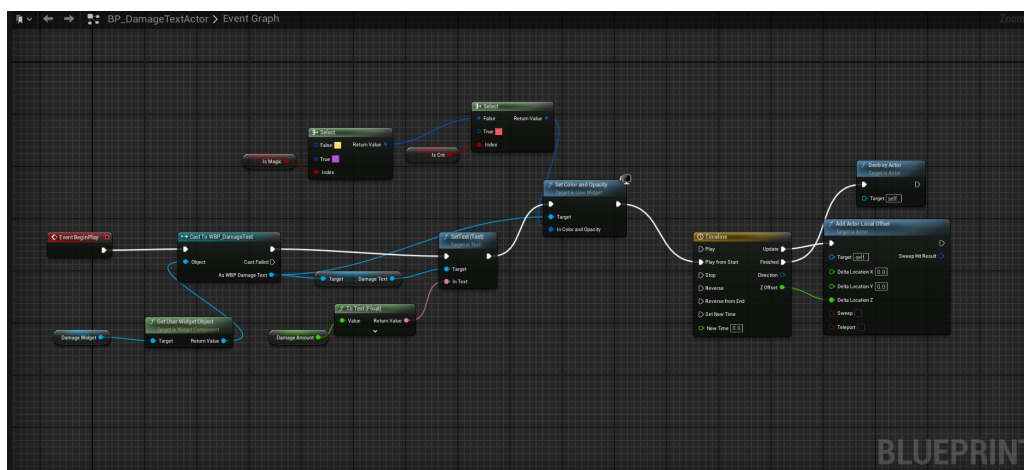


Figure D.6: Frontend logic for spawning and animating floating damage numbers during combat interactions to help visualise changes in damage stats when player buys items or when the stats get recalculated.



Figure D.7: Ingame view of the interactive Shopkeeper menu, displaying available equipment and items.

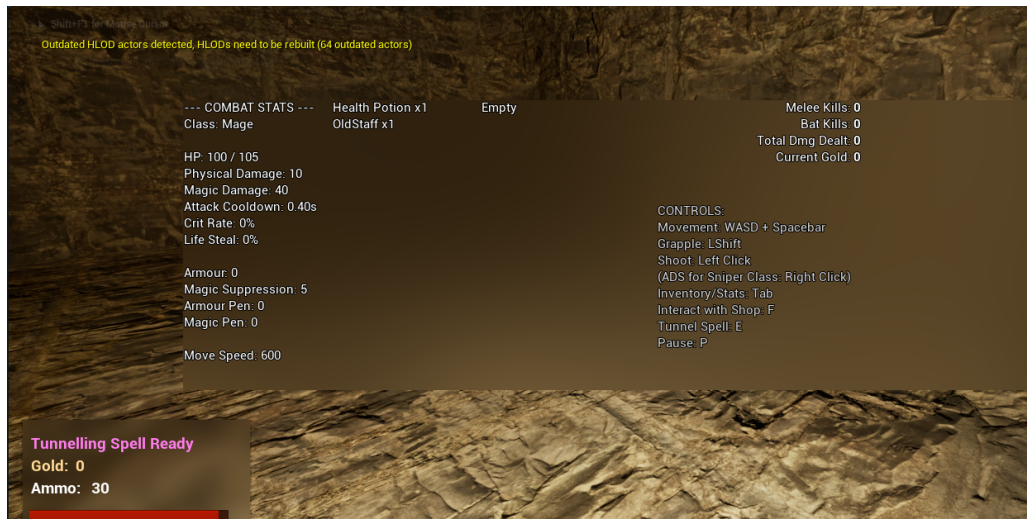


Figure D.8: Ingame view of the player's Inventory screen dynamically displaying current combat statistics, equipped items, collected materials, and kill counts/current gold for the current dungeon run.

E. Test Plans and Results

The following table details the white-box testing conducted to verify the core procedural generation algorithms and the Axis-Aligned Bounding Box (AABB) spatial logic.

Table E.1: White-Box Test Cases for Voxel Generation Logic

Test ID	Description	Expected Outcome	Actual Outcome	Result
TC-01	Player casts Tunnelling Spell into solid, uncarved void.	Algorithm validates AABB space and carves a new room.	Room carved successfully. Enemies spawned.	Pass
TC-02	Player casts spell towards an existing, densely packed room network.	AABB validation fails. Fallback logic triggers.	Fallback triggered. Scanning logic found existing air.	Pass
TC-03	Fallback logic attempts to carve tunnel to existing air.	Jagged tunnel is carved connecting the player to the existing room.	Tunnel carved to exact intersection point.	Pass
TC-04	Player casts spell with insufficient ammo.	Operation is blocked immediately. No voxel calculations run.	Operation blocked. HUD flashes 'Insufficient Ammo'.	Pass
TC-05	Player attempts to carve while enemies are still active in the room.	Operation is blocked by <code>IsRoomClear()</code> check.	Operation blocked. HUD indicates locked spell.	Pass

F. User and Installation Guide

This section provides instructions for launching and testing the Voxel Layout Generator prototype executable.

Installation and Setup

1. Extract the provided .zip archive containing the packaged project to a local directory on a Windows PC.
2. Ensure your system meets the minimum requirements (a modern multi-core CPU is strongly recommended for stable asynchronous voxel carving to actually work).
3. Navigate to the extracted folder and locate the executable file named `ProceduralDungeon.exe`.
4. Double-click `ProceduralDungeon.exe` to launch the application. No external dependencies or installations are required.

Basic Controls

- **W, A, S, D:** Move character
- **Mouse:** Look around / Aim
- **Left Click:** Primary Attack (Melee or Ranged depending on class)
- **Right Click:** Aim Down Sights (Sniper class only)
- **E:** Cast Tunnelling Spell too generate new tunnels or dungeon rooms(Requires ammo and the most recent dungeon room opened to be cleared of enemies)
- **F:** Interact (e.g., open Shopkeeper UI)
- **Spacebar:** Jump
- **L Shift:** Fire Grappling Hook
- **Tab:** Open Player Stats / Inventory Menu
- **H:** Use Health Potion

G. Generative AI Declaration

In accordance with the university guidelines on the use of Generative Artificial Intelligence, this appendix details the extent to which AI tools were used during the development of this project report.

Tool Used: Google Gemini

No AI tools were used to generate the core C++ codebase, blueprint logic, or raw performance data. All technical implementation and empirical testing results are the sole original work of the author. Gemini was used strictly as a formatting, research, and proofreading assistant, as detailed below:

1. LaTeX Environment Setup and Formatting

- **Prompt/Input:** "I have no prior knowledge of LaTeX. How do I download it and create a basic dissertation report template with a title page and chapters based on this project handbook ill provide so that I know if I've written too much in sections like the Output Summary?" **AI Output:** If you have no prior knowledge of LaTeX, the easiest way to start is by using a cloud-based editor called Overleaf...

```
1 \documentclass{report}
2
3 % --- Packages ---
4 \usepackage{geometry} % Sets standard 2.5cm margins
5 \usepackage{mathptmx} % Sets font to Times New
   Roman
6 \usepackage{graphicx} % Allows you to insert
   images
7 \usepackage{hyperref} % Creates clickable links
   in the PDF
8 \usepackage{natbib} % Enables Harvard-style
   referencing
9
10 \begin{document}
11
12 % --- Title Page ---
13 \begin{titlepage}
```

```
14 \centering
15 \vspace*{2cm}
16 {\Huge \textbf{Your Project Title Here}\par}
17 \vspace{1.5cm}
18 {\Large Final Year Project Report\par}
19 \vspace{1.5cm}
20 {\large \textbf{Student Name:} John Doe\par}
21 {\large \textbf{Consultant:} Dr. Jane Smith\par}
22 \vspace{2cm}
23 {\large BSc (Hons) Computer Science\par}
24 {\large City St George's, University of London\par}
25 \vfill
26 {\large \today\par}
27 \end{titlepage}
28
29 % --- Abstract ---
30 \begin{abstract}
31     This is where you write your abstract. It should be a
32         concise summary of the problem you addressed, the
33         methods you used, and your final results. (Typically
34         100-250 words).
35 \end{abstract}
36
37 % --- Table of Contents ---
38 \tableofcontents
39 \clearpage
40
41 % --- Main Body Chapters ---
42 \chapter{Introduction}
43 \section{Project Objectives}
44 This section outlines the primary technical goals and
45 requirements necessary to solve the problem.
46
47 \chapter{Output Summary}
48 \section{Project Deliverables}
49 This section provides a summary of the software and technical
50 artefacts produced during the project.
```

```
50 This section positions the project within the broader field
    of computer science and reviews existing commercial
    software.
51
52 \chapter{Method}
53 \section{Software Engineering Approach}
54 This section outlines the iterative development cycle and the
    system architecture chosen for the project.
55
56 \chapter{Results}
57 \section{Implementation and Testing Outcomes}
58 This section evaluates the functional success of the
    prototype and presents performance profiling metrics.
59
60 \chapter{Conclusions and Discussion}
61 \section{Objectives Review}
62 This section systematically evaluates the final prototype
    against the primary objectives established at the
    project's outset.
63
64 % --- References ---
65 % This tells LaTeX to use the Harvard referencing style and
    look for a file called 'references.bib'
66 \bibliographystyle{plainnat}
67 \bibliography{references}
68
69 % --- Appendices ---
70 \appendix
71 \chapter{Project Definition Document}
72 \section{Original Proposal}
73 Insert the text or PDF of your approved Project Definition
    Document here.
74
75 \end{document}
```

- **Action Taken:** Gemini provided instructions on using LaTeX and generated a standard template based on the Project Handbook to ensure I do not go over the word limit or page limit for certain sections. Although I used this template, I changed and added a lot of subsections and sections by myself so this was mainly used as a way for me to learn how to write academic papers in LaTeX and to ensure that the write syntax was used.

2. Word Count and Constraint Checking

- **Prompt/Input:** "I attached my up to date PDF for my project and the handbook can you give me the exact word count and see if it matches the guidelines so i dont go over or under the limits for anything"
- **Output:** "Word Count Analysis and Guideline Check Based on the document and handbook provided, here is the breakdown of your word count against the specific IN3007/INM450 guidelines for a BSc final year project."

1. Main Report Word Count: words

Status:

Guideline Check: The handbook explicitly states that the main body of a BSc report must be between 7,000 and 10,000 words (Section 8.1.4). This count excludes the title page, table of contents, abstract, glossary, references, and appendices.

Penalty Warning: If your final count exceeds 10,000 words, you will be penalized 1% for every 500 words over the limit. Projects exceeding 12,000 words face severe penalties. Ensure all supplementary material is pushed to the appendices.

2. Section-by-Section Breakdown While the global limit is 10,000 words, the handbook suggests specific lengths for your chapters to ensure proper balance:

Chapter 3 (Context): The recommendation is a maximum of 1,000 words. Your current count is ****.

Chapter 4 (Method): The recommendation is approximately 2,500 words. Your current count is ****.

Chapter 5 (Results): The recommendation is that this chapter should make up roughly half of your report (around 3,500 to 5,000 words). Your current count is ****.

3. Project Definition Document (Appendix A)

Guideline Check: The PDD has a strict maximum limit of 1,700 words for BSc students (excluding the cover sheet, references, ethics checklist, and client info sheet).

Current Count: **** words.

Recommendations: If you are close to the upper limit, ensure that you are not placing heavy code snippets or raw interview data in the main Method or Results chapters. The handbook states that all raw data, large UML diagrams, and complete software code must be placed in the appendices (Section 8.2.10) to preserve your main word count for critical analysis and discussion."

- **Action Taken:** Gemini provided the exact word count of the main content from the PDF and confirmed it was within the bounds of the project guidelines. This was used strictly to

track document length and ensure compliance with the Project Handbook. No report text was generated during this process. This function for tracking the word count and limit was used a few times.

3. Initial Literature and Context Research

- **Prompt/Input:** "What are examples of commercial games that actually use procedural content generation and voxel destructibility and how do their approaches differ"
- **Output:** "Examples of Commercial Games Using PCG and Voxel Destructibility
The intersection of Procedural Content Generation (PCG) and voxel-based destruction is handled differently across the commercial games industry, primarily divided by how the engines render the volumetric data and when the generation occurs.
1. *Minecraft*
Generation Approach: Uses continuous, seed-based procedural generation tied to player movement (chunk loading). It heavily relies on 3D Perlin noise and fractal algorithms to generate infinite macro-terrain.
Destructibility: Absolute micro-destruction.
Distinction: The voxel data is visually rendered as discrete 3D cubes. While players can destroy anything, they cannot trigger the dynamic generation of complex, structured rooms on demand; the generation is passive and environmentally driven.
2. *Deep Rock Galactic*
Generation Approach: Uses seed-based macro-generation to create massive, structured cave networks. However, the generation happens entirely during the loading screen rather than continuously during gameplay.
Destructibility: Absolute micro-destruction.
Distinction: Unlike *Minecraft*, it uses algorithms like Marching Cubes or Dual Contouring to wrap the raw voxel data in a smooth polygonal mesh. This allows players to dynamically carve organic, smooth tunnels through the terrain in real-time without the environment looking blocky.
3. *Noita*
Generation Approach: Uses a combination of rigid pre-authored layouts and procedural generation to build its world.
Destructibility: Every single pixel on the screen is physically simulated.
Distinction: It uses a "Falling Sand" engine based on cellular automata rather than a 3D volumetric grid. Because it is strictly constrained to 2D, it can afford the $\mathcal{O}(n^2)$ computational cost of simulating individual pixel physics and chemical reactions, something that would crash a 3D engine.
4. *Enshrouded*
Generation Approach: Relies heavily on a massive, handcrafted open world rather than pure PCG, but the entire world is constructed using an underlying voxel framework.
Destructibility: High-fidelity micro-destruction.
Distinction: It uses incredibly small "micro-voxels" integrated into a custom engine. This allows for hyper-realistic graphics and lighting while still letting players carve organic holes into mountainsides or destroy buildings block-by-block."
- **Action Taken:** Gemini suggested titles such as *Minecraft*, *Noita*, *Hades*, and *Dead Cells*, providing a brief summary of how each handles generation (such as room-based PCG

vs. cellular automata). I used these game titles as a starting point to conduct my own in depth academic research, allowing me to structure the "Existing and Related Work" section (Section 3.2) meaning that I did not blindly follow what Gemini produced but instead used it as a search tool rather than a generative tool in order to save time.

4. LSEPI Framework Structuring

- **Prompt/Input:** "What are the Legal, Social, Ethical, and Professional Issues (LSEPI) I should consider for a single-player, offline prototype unreleased game that features voxel combat"
- **Output:** LSEPI Considerations for an Offline Voxel Prototype For a single-player, offline prototype involving voxel-based combat and procedural generation, the Legal, Social, Ethical, and Professional Issues (LSEPI) are focused primarily on intellectual property, user safety, and professional standards.

1. Legal Issues

Intellectual Property (IP): The use of third-party voxel plugins or assets must comply with their specific licenses (e.g., MIT, LGPL). Attribution is required for all open-source code used in the layout generator.

Copyright: Since voxels are rendered as polygonal meshes, ensure no copyrighted character designs or textures are replicated within the prototype.

2. Social Issues

Accessibility: Procedurally generated caves can present navigation challenges. Socially responsible design should consider high-contrast materials for voxels to assist visually impaired users and ensure the generation algorithm does not create "soft-locks" (areas the player cannot escape).

Representation of Violence: While voxel combat is often abstract (low-fidelity), the social impact of combat mechanics should be considered, ensuring the "destruction" remains aligned with the intended gameplay age rating.

3. Ethical Issues

Addictive Loops: Procedural generation (PCG) can create "infinite" gameplay loops. Ethically, a developer should ensure the game does not rely on dark patterns or compulsive reward cycles, even in an offline prototype.

Safety and Warnings: Because the engine allows for carving any surface, the prototype must ensure that high-frequency patterns or flashing lights caused by rapid voxel destruction do not trigger photosensitive epilepsy.

4. Professional Issues

Code Quality and Standards: Following the Epic C++ Coding Standard is a professional obligation to ensure the codebase is maintainable and transparent for future developers.

Competence: In accordance with the BCS Code of Conduct, the developer must be transparent about the limitations of the prototype (e.g., acknowledging that it is a "proof of concept" and not a commercially hardened product ready for general distribution).

- **Action Taken:** I researched more upon the possible LSEPI issues that were mentioned on the internet and I used these concepts as a conceptual framework to write my own LSEPI analysis in Section 6.5, ensuring I covered all the necessary professional and legal angles required by the BCS Code of Conduct.

No AI tools were used to generate the core C++ codebase, blueprint logic, or raw performance data. All technical implementation and empirical testing results are the sole original work of the author.